

8-Bit Successive Approximation Register (SAR) ADC: Exploring Mixed-Signal Design with Xschem

Phillip Ferreira Baade–Pedersen (Design Engineer)
Erlangen - 24/02/2026

Projects: BMBF -> FMD-QNC (16ME0831)

Key Takeaways from Yesterday



- 0 Overview of Qucs-S
- 0 Understanding Issues with Xyce/QUCS-S
- 0 Electromagnetic (EM) Simulation
- 0 Developing RF Workflow Skills in OS

Questions?

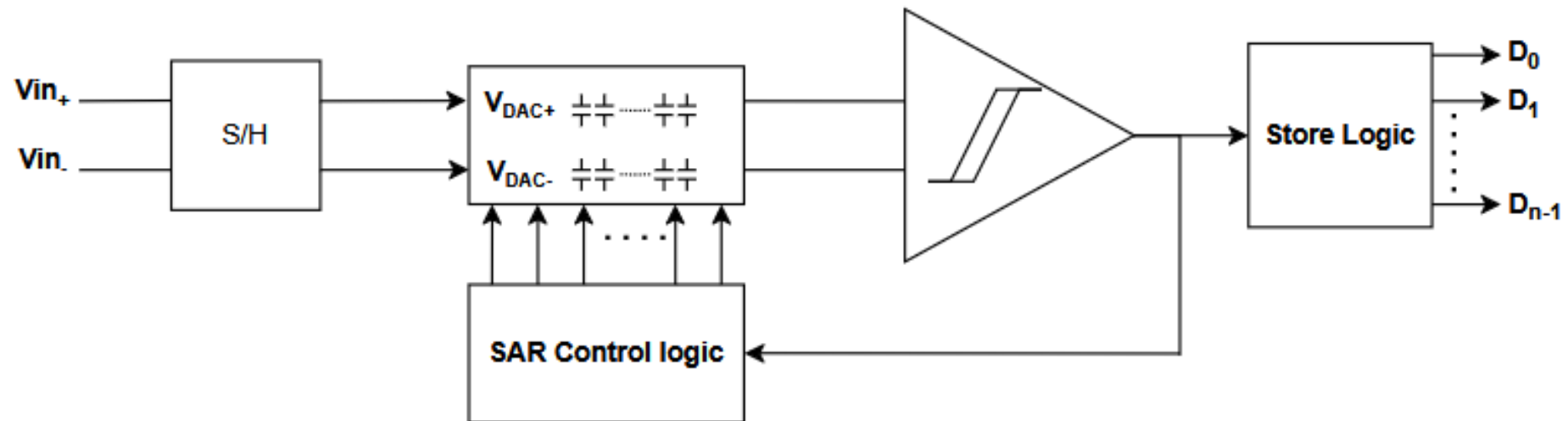
- 0 Was something from yesterday unclear?
- 0 Are we going to fast?
- 0 Do you need more time for the small exercise?

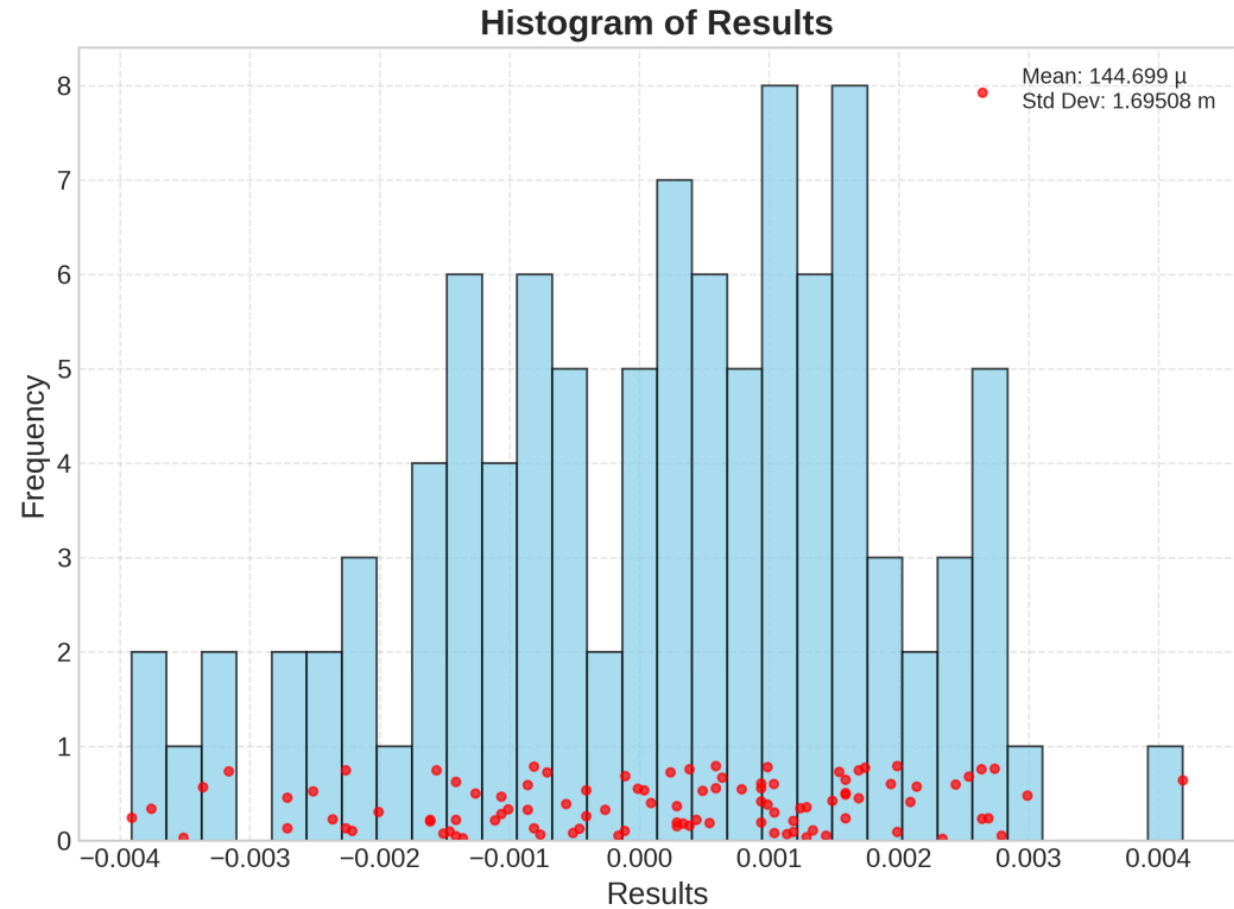
Overview Of Todays Plan



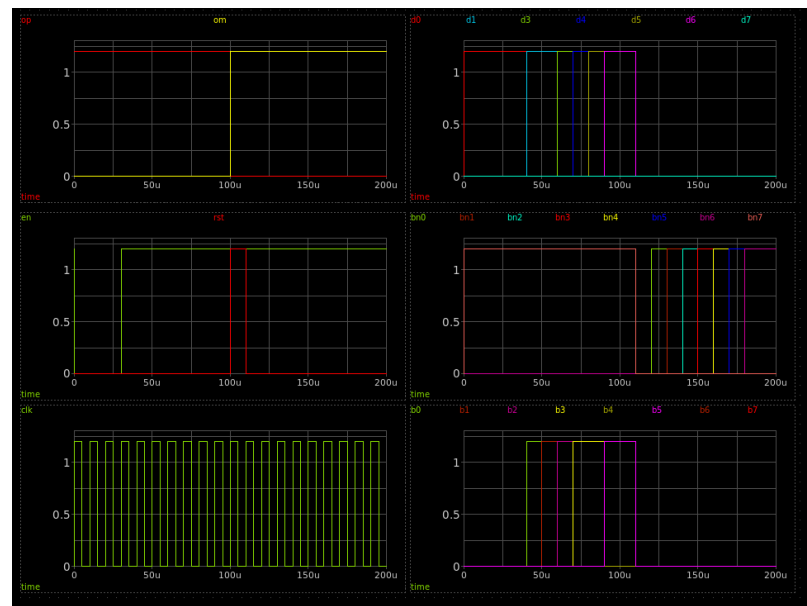
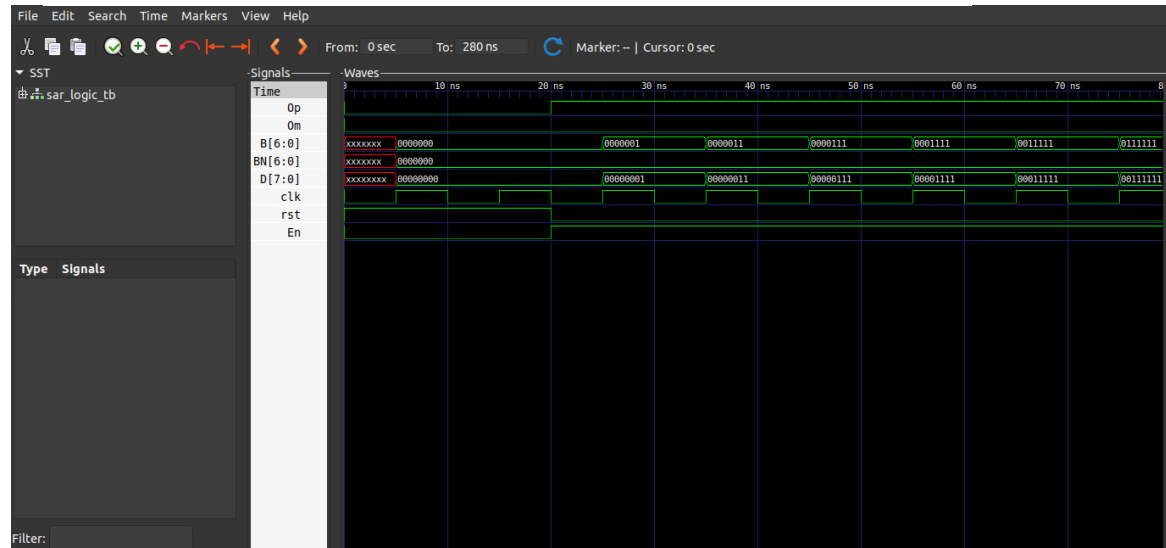
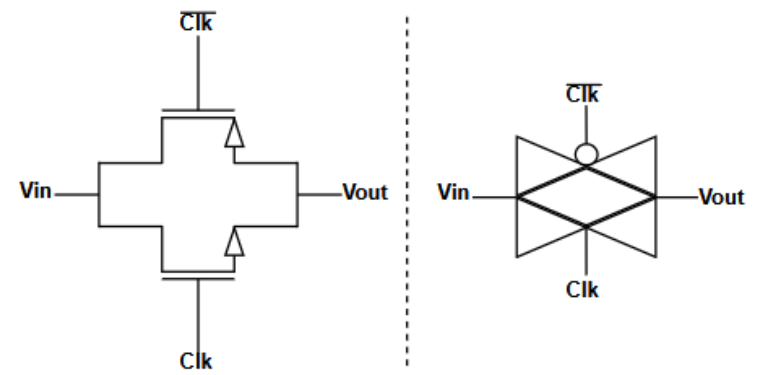
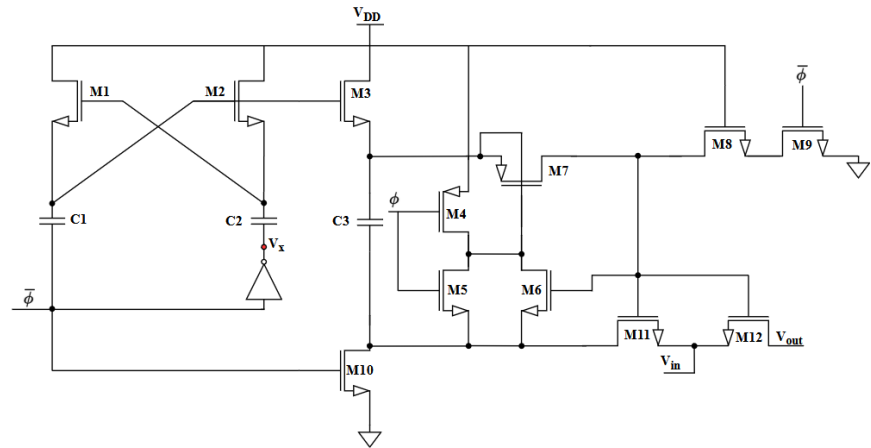
- 0 **Review of Key Skills:** Applying knowledge from previous sessions to create small circuits for the SAR ADC, including Monte Carlo analysis.
- 0 **Digital Simulations:** Simulating Verilog algorithm
- 0 **AMS Workflow:** Establishing an Analog-Mixed Signal (AMS) workflow using Xschem
- 0 **Misc:** Continuing with the layout of the OTA

The SAR ADC





Digital Components & S/H





Leibniz Institute
for high
performance
microelectronics

Digital Integration In Xschem

Some open source EDA tools – digital design



Simulation	Synthesis	PnR	GDS	Verification
Iverilog	Yosys	OpenROAD	Klayout	cocotb
Verilator		Coriolis	Magic	pyUVM
Ghdl				

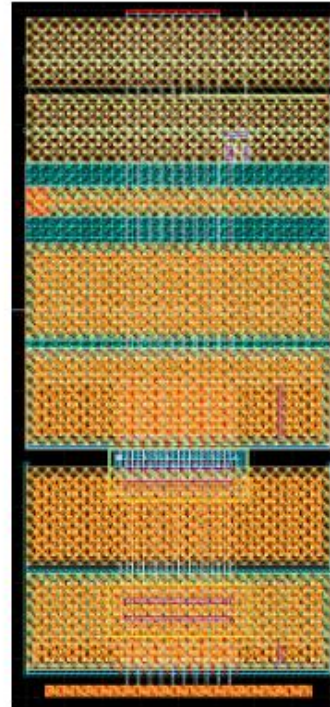
```
always @(posedge clk) begin
  if (rst) begin
    done_q <= 1'b0;
    bit_ct_q <= 3'b0;
    dout_q <= 8'b0;
    miso_q <= 1'b1;
  end else begin
    done_q <= done_d;
    bit_ct_q <= bit_ct_d;
    dout_q <= dout_d;
    miso_q <= miso_d;
  end
end
```



Digital primitives

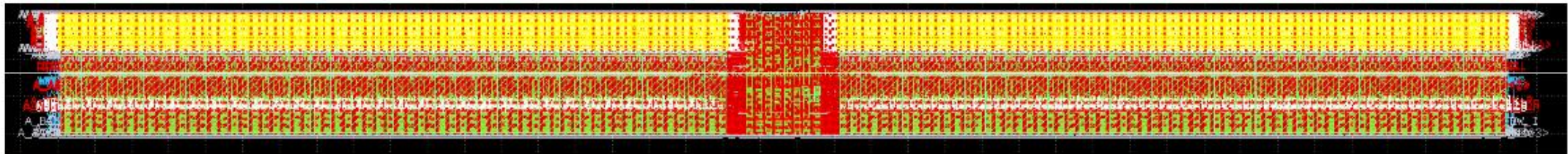
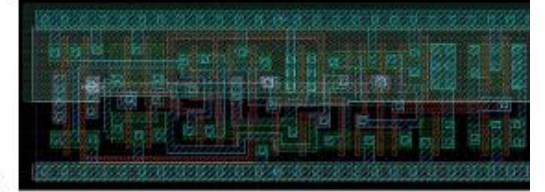


	StdCells	IOCells	SRAM
Verilog	X	X	X
LIB	X	X	X
LEF	X	X	X
CDL	X	X	X
SPICE	X	X	X
GDS	X	X	X



Digital components:

- stdcells
 - 78 cells,
 - combinatorial logic,
 - sequential elements,
 - scan flops,
 - gated cells.
- IOCells,
 - In, Out, InOut, Analog
 - different drive strengths
- SRAM
 - hard macros of a single port SRAM
 - different sizes
 - SRAM generator



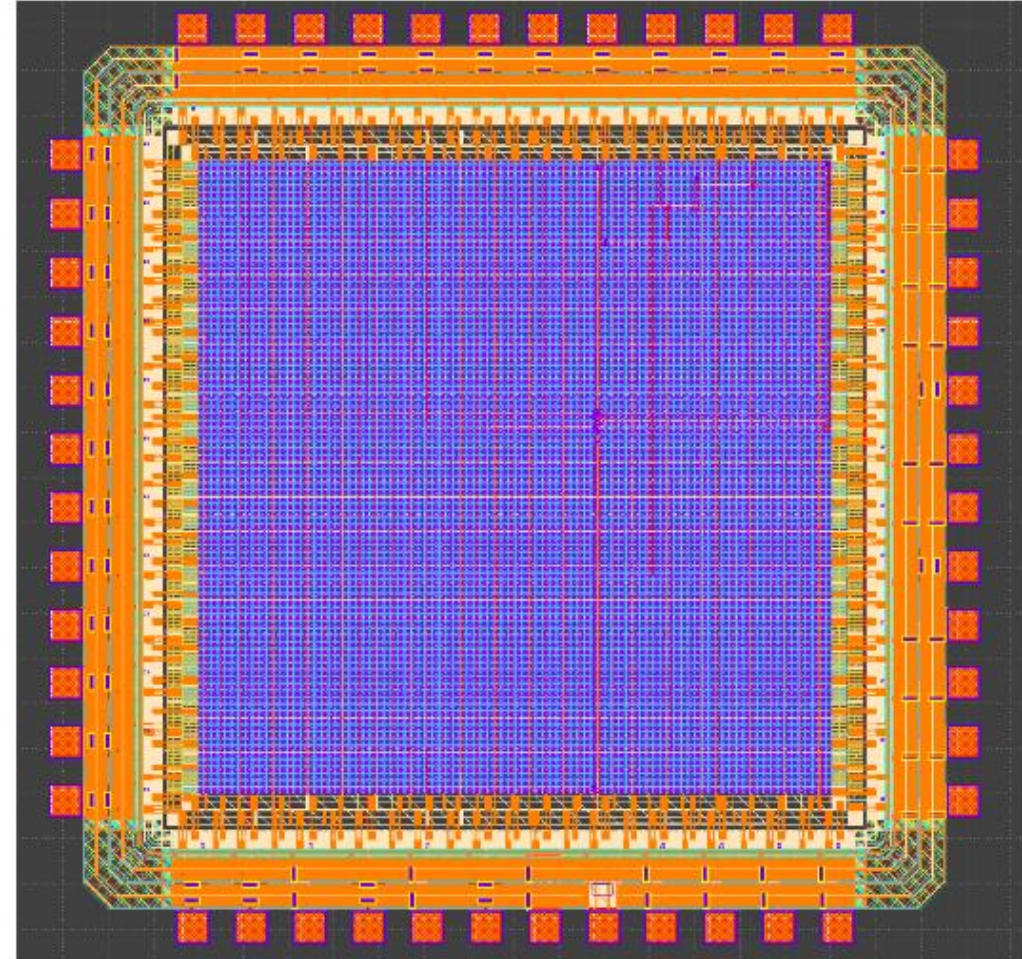
Mixed signal approach - goals



- Design a mixed signal chip
- Enable the simulation in two domains: analog and digital,
- Accurately model circuits behavior in both domains and at the interfaces.
- Automation of the design process

Key issues:

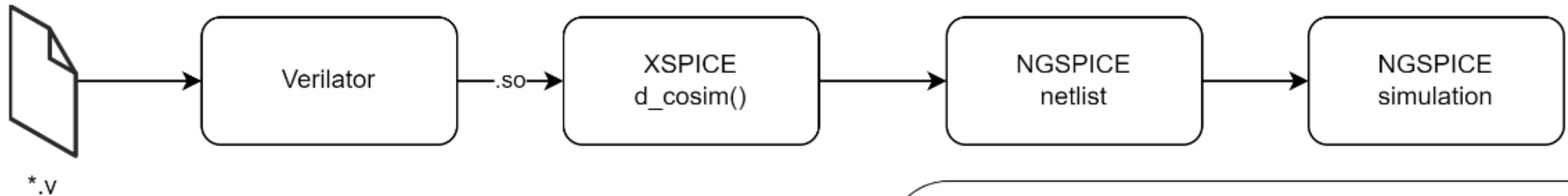
- Analog and digital behavioral models are handled differently
- Timing abstraction has to be handled as well



Mixed signal circuits development using OS tools



Event based acceleration in XSPICE, being part of the ngspice software

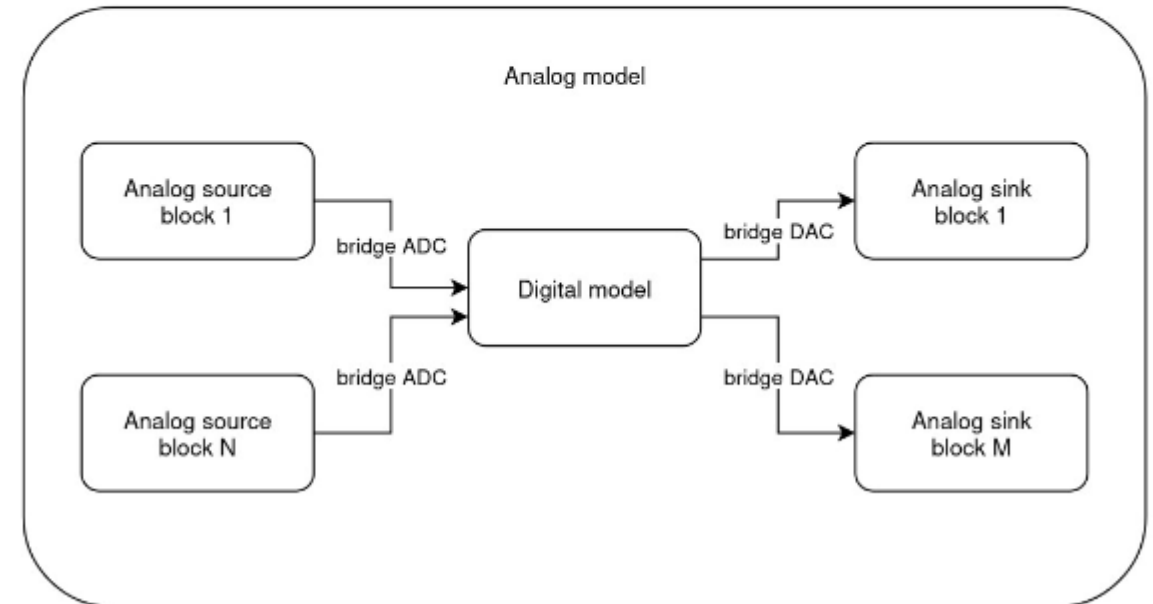


Many models of digital primitives, sources and sinks have been already modeled in XSPICE

Verilog and VHDL supported

Verilator provides an elegant and scalable solution for encapsulation of a digital module

Gnucap as a future „post-spice” era mixed signal circuit simulator



Part 1

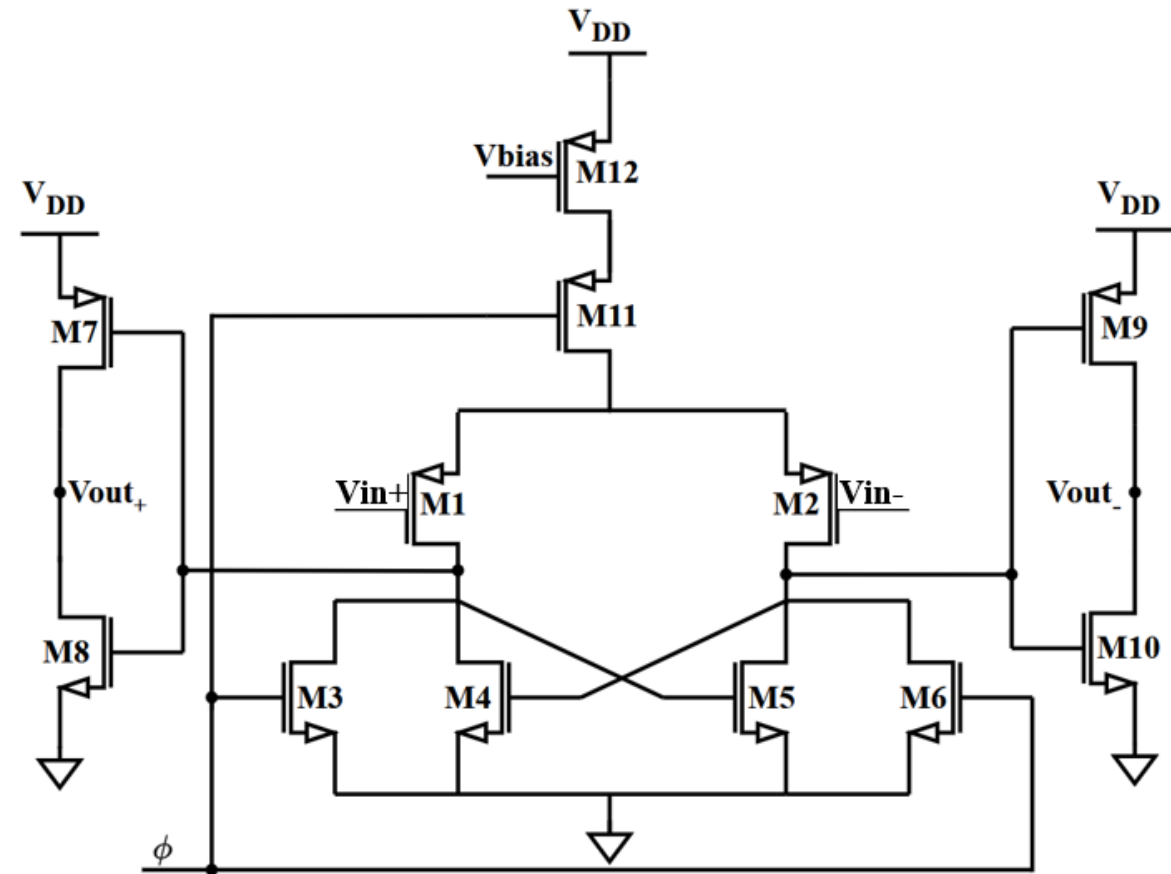
Comparator

Reference: [modules/module_3_8_bit_SAR_ADC/part_1_comparator](#)

Dynamic Comparator



- Heart of the SAR ADC
- Makes decisions on falling edge!
- We want to evaluate the offset of this comparator!



Dynamic Comparator



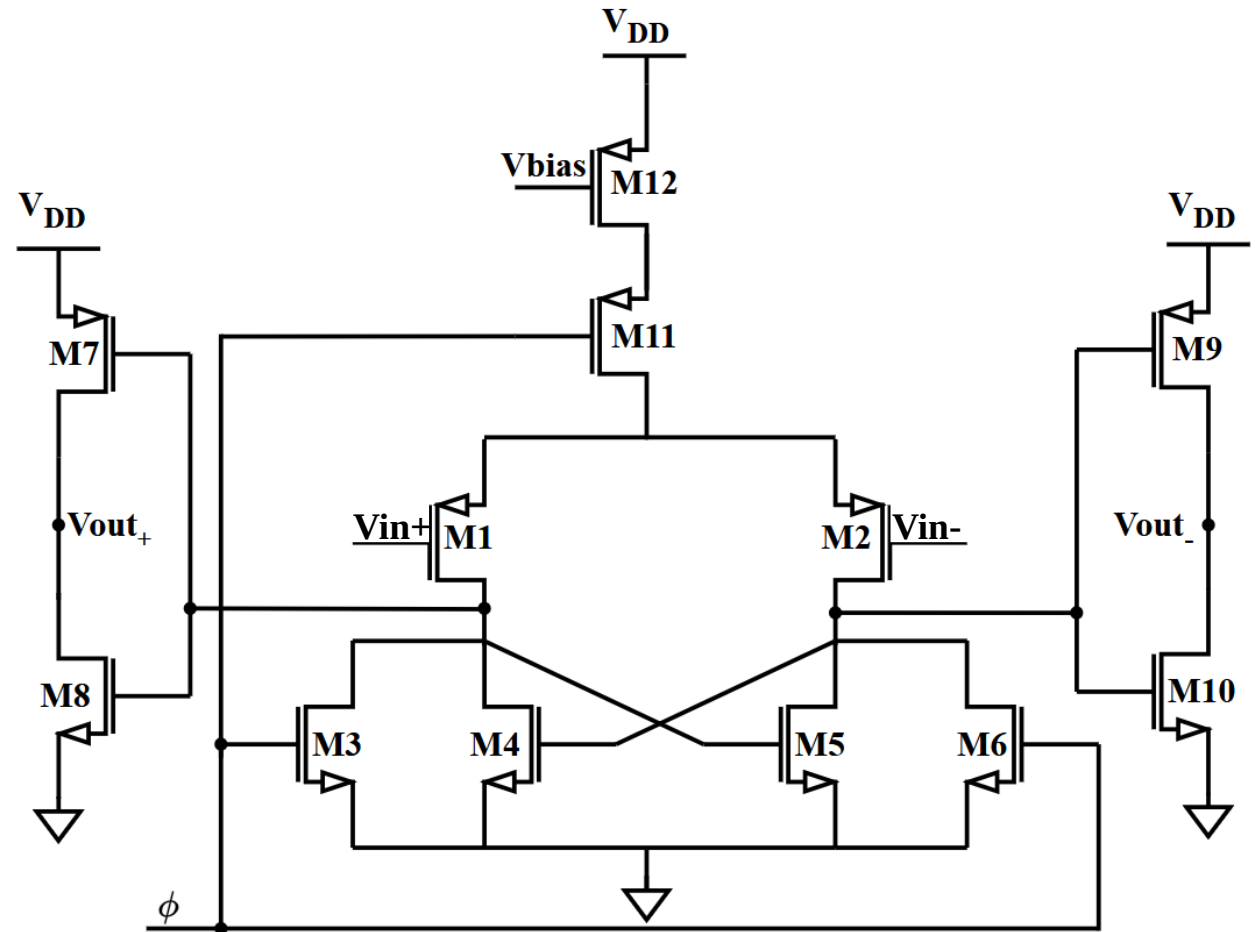
Specs

- 0 **V_{DD}** is set to **1.2V**
- 0 The **bias voltage** is set to **0.6V**
- 0 A clock signal can be defined as follows:

PULSE(V1 V2 TD TR TF PW PER NP)

Name	Parameter	Default Value	Units
V1	Initial value	-	V, A
V2	Pulsed value	-	V, A
TD	Delay time	0.0	sec
TR	Rise time	TSTEP	sec
TF	Fall time	TSTEP	sec
PW	Pulse width	TSTOP	sec
PER	Period	TSTOP	sec
NP	Number of Pulses *)	unlimited	-

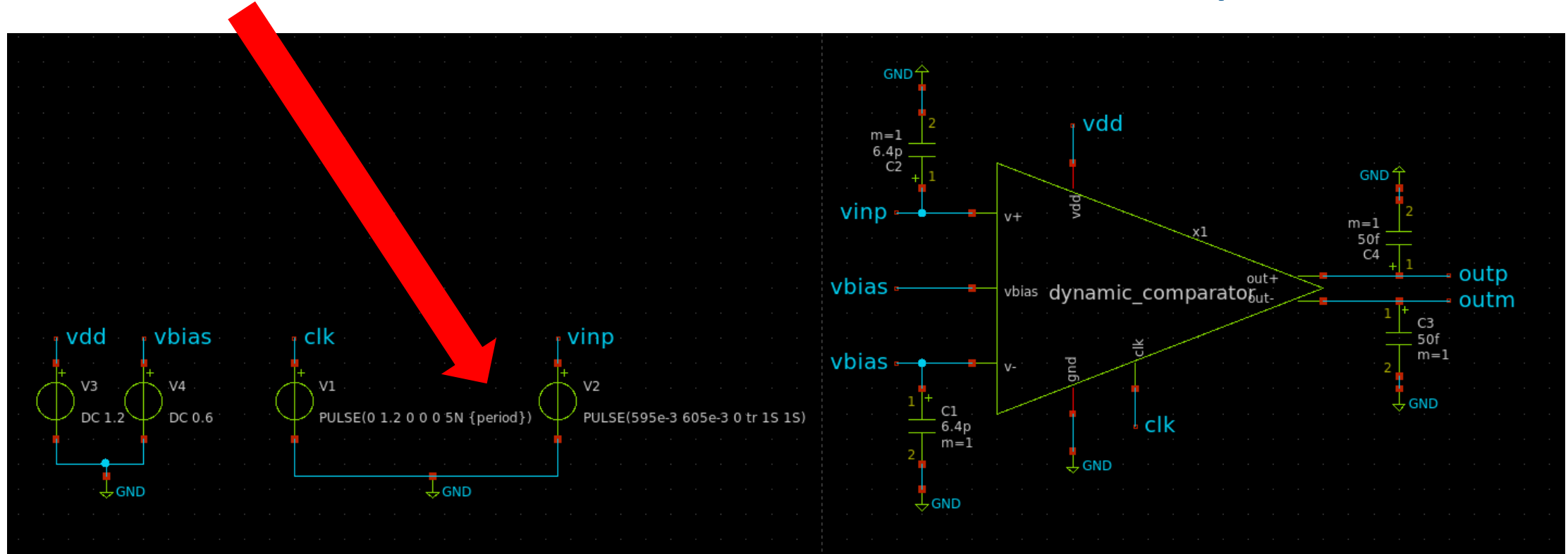
- **M1, M2:** $W = 32 \mu\text{m}$, $L = 0.2 \mu\text{m}$, $ng = 4$
- **M3, M4, M5, M6:** $W = 4 \mu\text{m}$, $L = 0.2 \mu\text{m}$
- **M8, M10:** $W = 4 \mu\text{m}$, $L = 0.2 \mu\text{m}$
- **M7, M9:** $W = 8 \mu\text{m}$, $L = 0.2 \mu\text{m}$
- **M11, M12:** $W = 18 \mu\text{m}$, $L = 0.3 \mu\text{m}$, $ng = 4$



Create The Testbench



Period is defined in simulation code block, and same for risetime in *vinp*

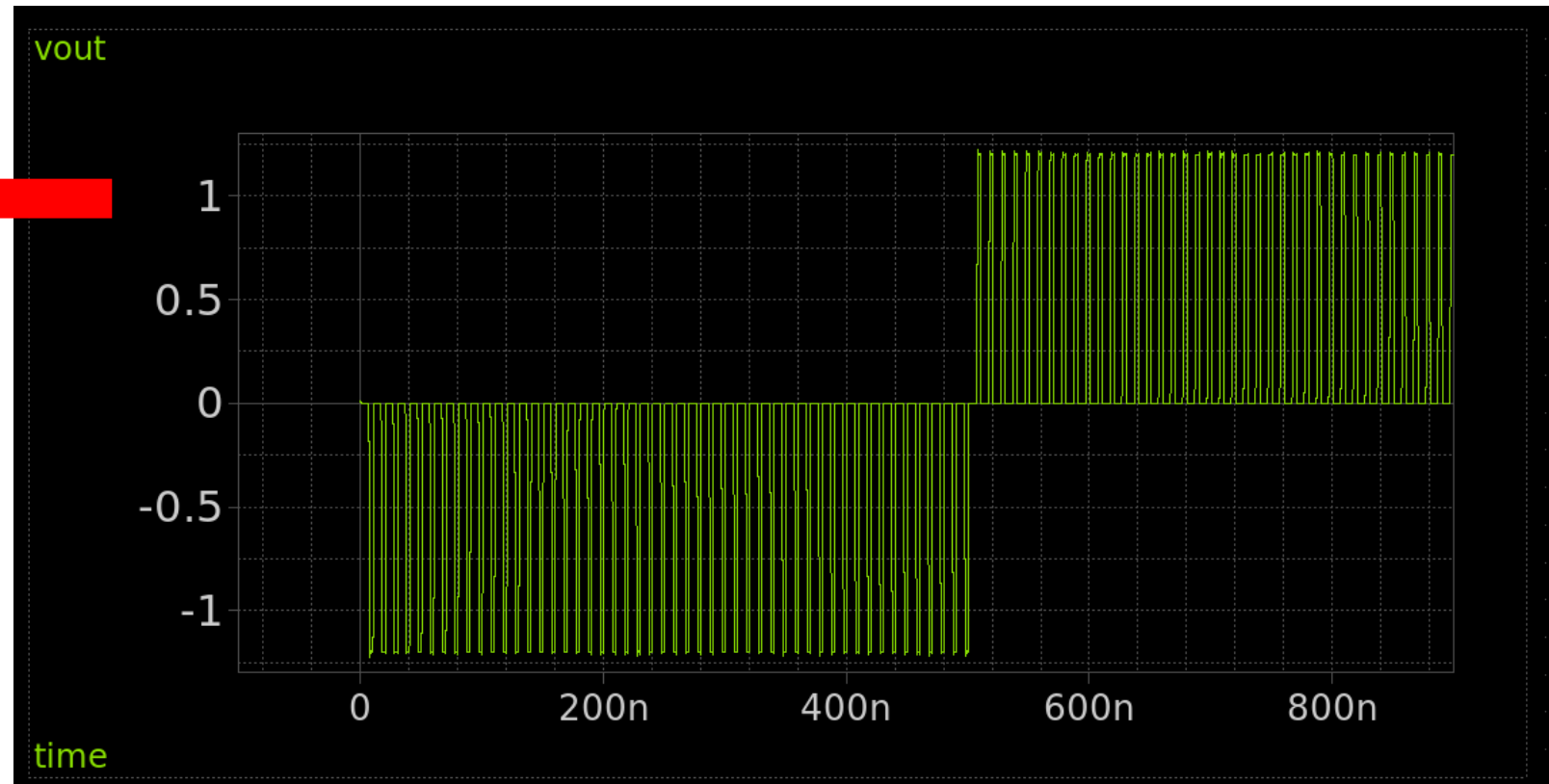


Plot the corresponding output from a transient analysis and verify the operation

Create The Testbench



```
name=NGSPICE only_toplevel=false
value=""
.control
op
.param clock = 100e6 ; 100 MHz clock
.param period = 1 / clock
.param num_cycles = 100 ; cycles
.param tr = num_cycles * period
tran 500p 1u
.save all
let vindiff = (v(vinp))-(v(vbias))
let clk = v(clk)
let vout = (v(outp))-(v(outm))
write output_file.raw
.endc
"
```





Leibniz Institute
for high
performance
microelectronics

Part 2

Digital Components

Reference: [modules/module_3_8_bit_SAR_ADC/part_2_digital_comps](#)

SAR Algorithm

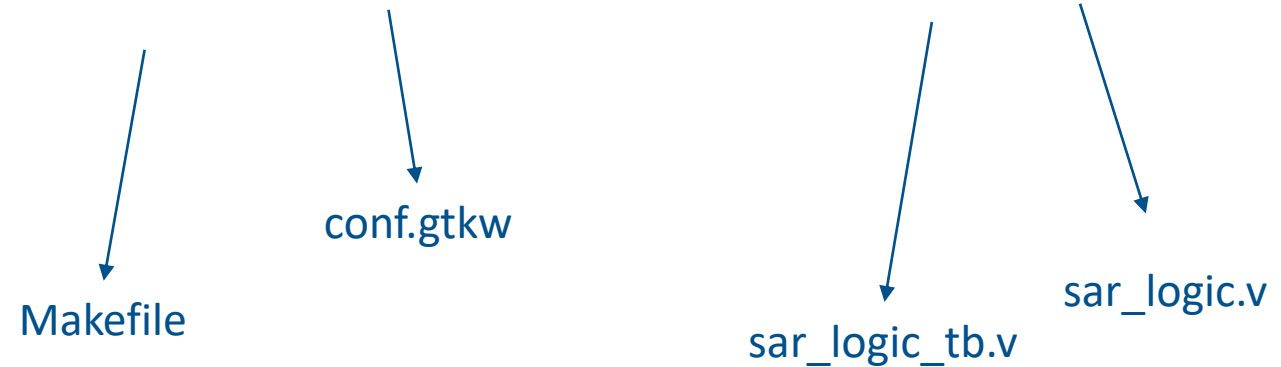


```
always @(posedge clk) begin
  if (rst) begin
    B <= 7'b00000000;
    BN <= 7'b00000000;
    D <= 8'b00000000;
    counter <= 4'b0000;
  end else if (En && (Op ^ Om)) begin
    if (counter < 7) begin
      D <= D | ({7'b0, Op} << counter);

      B[counter % 7] <= (Op) ? 1'b1 : 1'b0;
      BN[counter % 7] <= (Om) ? 1'b1 : 1'b0;

      counter <= counter + 1'b1;
    end
  end
end
```

module_3_8_bit_SAR_ADC/part_2_digital_comps/algorithm/verilog



The algorithm folder in the repository can be copied directly to your own directory for simplicity

Makefile



```
# Define variables
IVERILOG = iverilog
VVP = vvp
GTKWAVE = gtkwave
NGSPICE = ngspice
OUTPUT = sar_logic.out
MIXED = sar_logic.so
MIXED_OBJ = sar_logic_obj_dir
VCD_FILE = sar_logic_tb.vcd
SRC_FILES = sar_logic.v sar_logic_tb.v
NGSPICE_FILE = sar_logic.v
DEST_DIR = ../xschem/simulations
```



```
# Default target
all: run_wave

# Compile using iverilog
$(OUTPUT): $(SRC_FILES)
    $(IVERILOG) -o $(OUTPUT) $(SRC_FILES)

# Run the simulation with vvp
run: $(OUTPUT)
    $(VVP) $(OUTPUT)

copy:
    cp $(MIXED) $(DEST_DIR)

# Open the wave file in GTKWave
wave: $(VCD_FILE)
    $(GTKWAVE) $(VCD_FILE) conf.gtkw

# Run ngspice vlnnggen command
ngspice:
    $(NGSPICE) vlnnggen $(NGSPICE_FILE)

# Execute both the simulation and wave viewer
run_wave: run wave

# Run ngspice and then the rest of the steps
full: ngspice run_wave copy

# Clean up generated files
clean:
    rm -f $(OUTPUT) $(VCD_FILE) $(MIXED)
    rm -rf $(MIXED_OBJ)
```



make run
make copy
make wave
make ngspice
make run_wave
make full
make clean

```
[*]
[*] GTKWave Analyzer v3.3.116 (w)1999-2023 BSI
[*] Mon Sep 30 13:11:15 2024
[*]
[dumpfile] "sar_logic_tb.vcd"
[dumpfile_mtime] "Mon Sep 30 13:10:47 2024"
[dumpfile_size] 777
[savefile] "conf.gtkw"
[timestart] 0
[size] 1906 985
[pos] -1 -1
*-14.000000 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1
[sst_width] 269
[signals_width] 102
[sst_expanded] 1
[sst_vpaned_height] 291
@28
sar_logic_tb.Op
sar_logic_tb.Om
sar_logic_tb.B[6:0]
sar_logic_tb.BN[6:0]
sar_logic_tb.D[7:0]
sar_logic_tb.clk
sar_logic_tb.rst
sar_logic_tb.En
@29
[pattern_trace] 1
[pattern_trace] 0
```

Sets initial waves to be plotted when running the make file

```
module sar_logic (  
    input wire clk,  
    input wire Op,  
    input wire En,  
    input wire Om,  
    input wire rst,  
    output reg [6:0] B,    // 7-bit  
    output reg [6:0] BN,  // 7-bit  
    output reg [7:0] D    // 8-bit  
);  
  
reg [3:0] counter = 4'b0000; // 4-bit counter  
  
always @(posedge clk) begin  
    if (rst) begin  
        B <= 7'b0000000;  
        BN <= 7'b0000000;  
        D <= 8'b00000000;  
        counter <= 4'b0000;  
    end else if (En && (Op ^ Om)) begin  
        if (counter < 7) begin  
            D <= D | ({7'b0, Op} << counter);  
  
            B[counter % 7] <= (Op) ? 1'b1 : 1'b0;  
            BN[counter % 7] <= (Om) ? 1'b1 : 1'b0;  
  
            counter <= counter + 1'b1;  
        end  
    end  
end  
endmodule
```

SAR Logic Testbench



```
`timescale 1ns/1ps

module sar_logic_tb();

    // Inputs
    reg clk;
    reg rst;
    reg Op;
    reg Om;
    reg En;

    // Outputs
    wire [6:0] B;
    wire [6:0] BN;
    wire [7:0] D;

    // Instantiate the SAR Logic module
    sar_logic uut (
        .clk(clk),
        .rst(rst),
        .Op(Op),
        .Om(Om),
        .En(En),
        .B(B),
        .BN(BN),
        .D(D)
    );
endmodule
```

```
// Clock generation
always #5 clk = ~clk;

// Test sequence
initial begin
    $dumpfile("sar_logic_tb.vcd"); // Name of the VCD file
    $dumpvars(0, sar_logic_tb);   // Dump all variables in the testbench

    // Initialize inputs
    clk = 1'b0;
    rst = 1'b1;
    Op = 1'b0;
    Om = 1'b0;
    En = 1'b0;

    // Apply reset
    #20 rst = 1'b0;
    En = 1'b1;
    Op = 1'b1;
    Om = 1'b0;

    // Apply reset again
    #80 rst = 1'b1;
    #10 rst = 1'b0;
    Op = 1'b0;
    Om = 1'b1;

    #70 rst = 1'b1;

    // End of simulation
    #100 $finish;
end

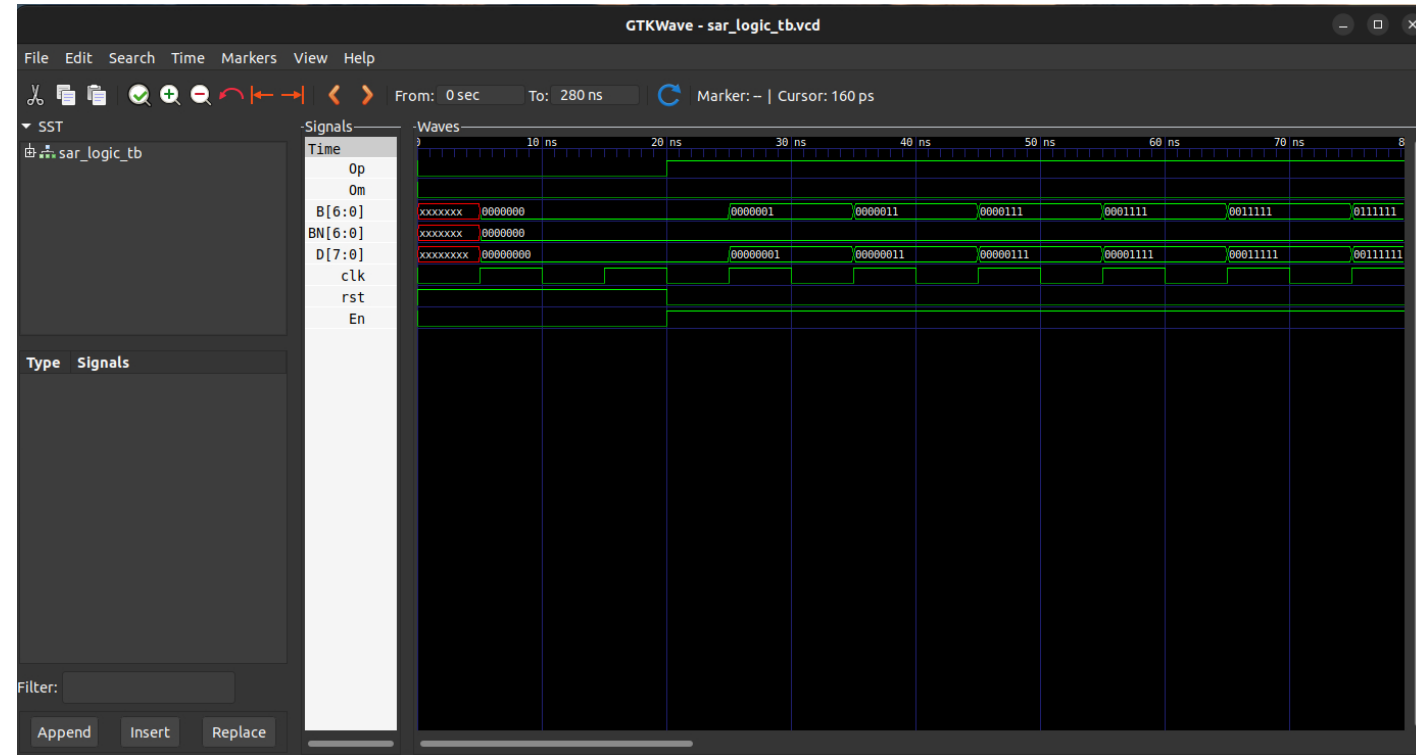
endmodule
```


Running Make full



cp sar_logic.so ../xschem/simulations
Why??

conf.gtkw
Makefile
sar_logic_obj_dir
sar_logic.out
sar_logic.so
sar_logic_tb.v
sar_logic_tb.vcd
sar_logic.v
sar_logic.vcd



Creating Symbol For The Digital Block



➡ `module_3_8_bit_SAR_ADC/part_2_digital_comps/algorithm/python`

→ `python git:(main) X python3 generate_sym.py`

Usage: `python generate_sym.py <verilog_file> <symbol_file>`

Example: `python generate_sym.py control.v test.sym`

Usage from Xschem directory:

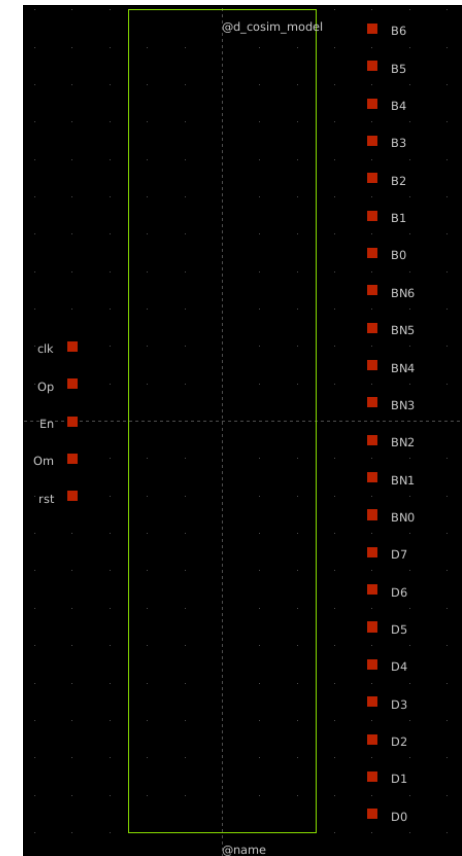
`python3 ../python/generate_sym.py ../verilog/sar_logic.v sar_logic.sym`

Parsed module: `sar_logic`

Inputs: ['clk', 'Op', 'En', 'Om', 'rst']

Outputs: ['D7', 'D6', 'D5', 'D4', 'D3', 'D2', 'D1', 'D0', 'BN6', 'BN5', 'BN4', 'BN3', 'BN2', 'BN1', 'BN0', 'B6', 'B5', 'B4', 'B3', 'B2', 'B1', 'B0']

Xschem symbol written to `sar_logic.sym`



Creating Symbol For The Digital Block



```
1 v {xschem version=3.4.5 file_version=1.2
2 *
3 * This file is part of XSCHEM,
4 * a schematic capture and Spice/Vhdl/Verilog netlisting tool for circuit
5 * simulation.
6 * Copyright (C) 1998-2023 Stefan Frederik Schippers
7 *
8 * This program is free software; you can redistribute it and/or modify
9 * it under the terms of the GNU General Public License as published by
10 * the Free Software Foundation; either version 2 of the License, or
11 * (at your option) any later version.
12 *
13 * This program is distributed in the hope that it will be useful,
14 * but WITHOUT ANY WARRANTY; without even the implied warranty of
15 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
16 * GNU General Public License for more details.
17 *
18 * You should have received a copy of the GNU General Public License
19 * along with this program; if not, write to the Free Software
20 * Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
21 }
22 G {}
23 K {type=delay
24 verilog_ignore=true
25 vhdl_ignore=true
26 format="@name [ @@clk @@Op @@En @@Om @@rst ] [ @@B6 @@B5 @@B4 @@B3 @@B2 @@B1 @@B0 @@BN6 @@BN5 @@BN4 @@BN3 @@BN2 @@BN1 @@BN0 @@D7 @@D6 @@D5 @@D4 @@D3 @@D2 @@D1 @@D0 ] null @dut
27 .model @dut @d_cosim_model simulation=@model"
28 template="name=adut
29 dut=dut
30 d_cosim_model= d_cosim
31 model=./sar_logic.so" }
32 V {}
33 S {}
34 E {}
```

Creating Symbol For The Digital Block



Stefan Schippers, the creator of Xschem, has made an in-depth video demonstrating how to create symbols for co-simulation. He also explains how to generate multidimensional outputs, which helps avoid large and cluttered symbol structures like the one in our example. Instead of manually expanding each signal, they can be represented more compactly—for example, as:

D[6..0]

<https://www.youtube.com/watch?v=PPd7jkcHOgA>

For this tutorial a script to generate such a symbol for our use-case specifically was made to save time!!

Digital Block In Xschem

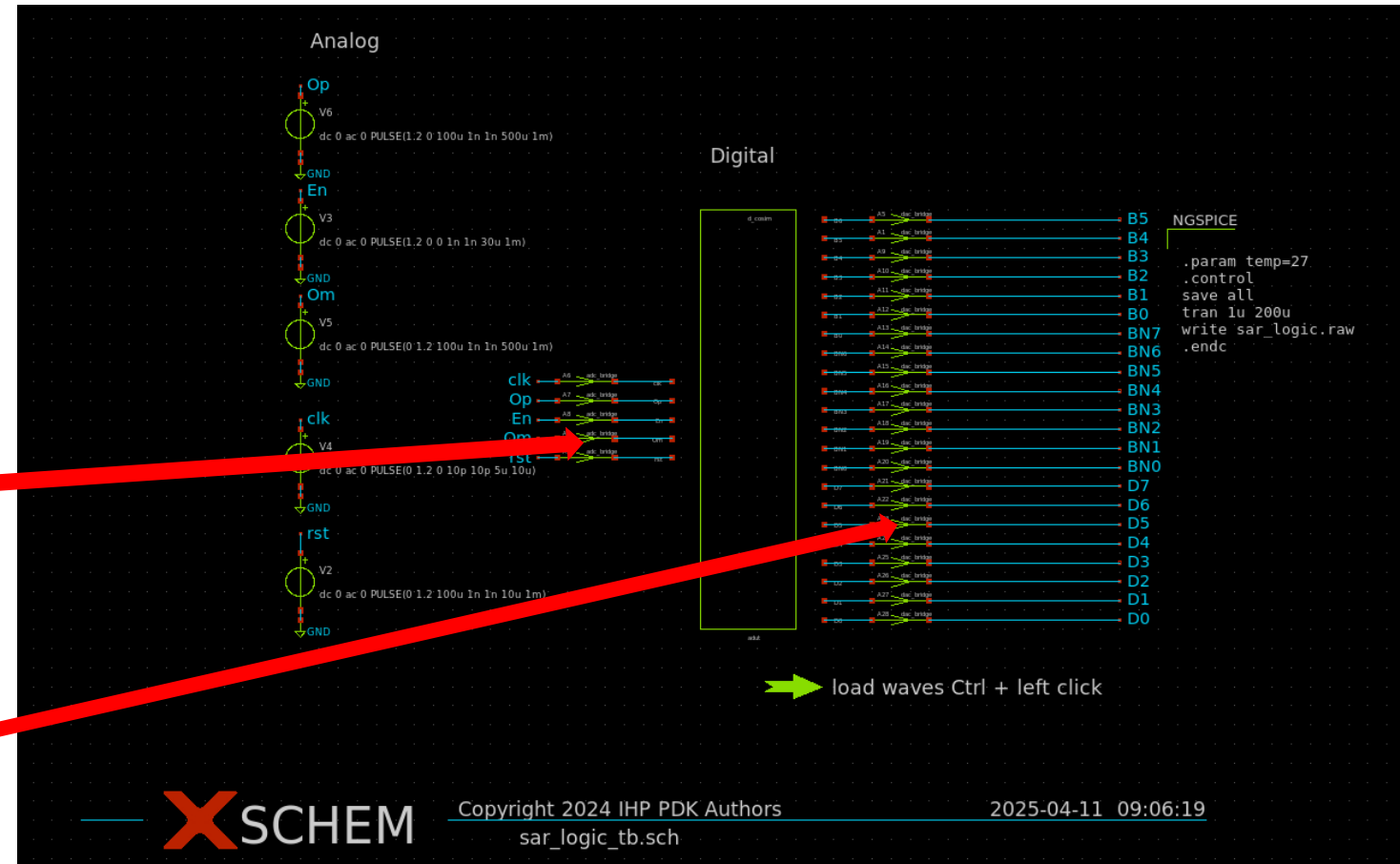
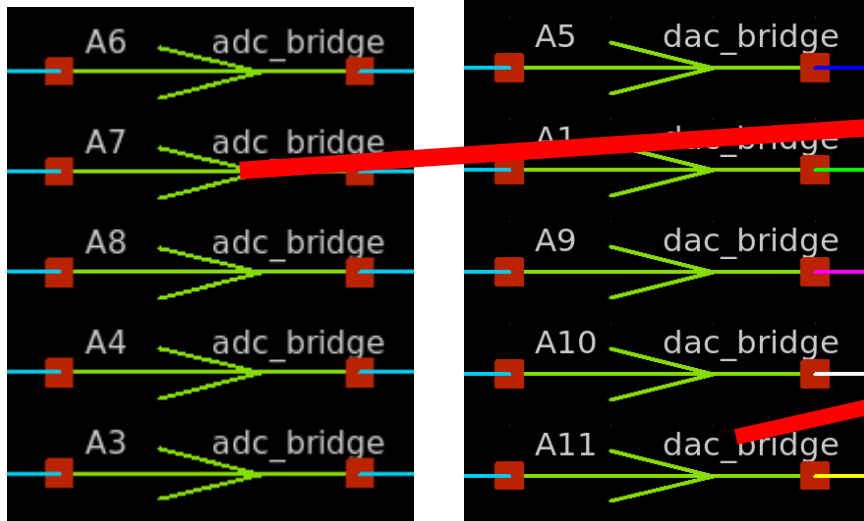


Navigate to the following path:

module_3_8_bit_SAR_ADC/part_2_digital_comps/algorithm/xschem

→ xschem git:(main) X xschem sar_logic_tb.sch

We need adc_bridge and dac_bridge to interface with the digital block (can be found in same directory as the testbench)



Digital Block In Xschem



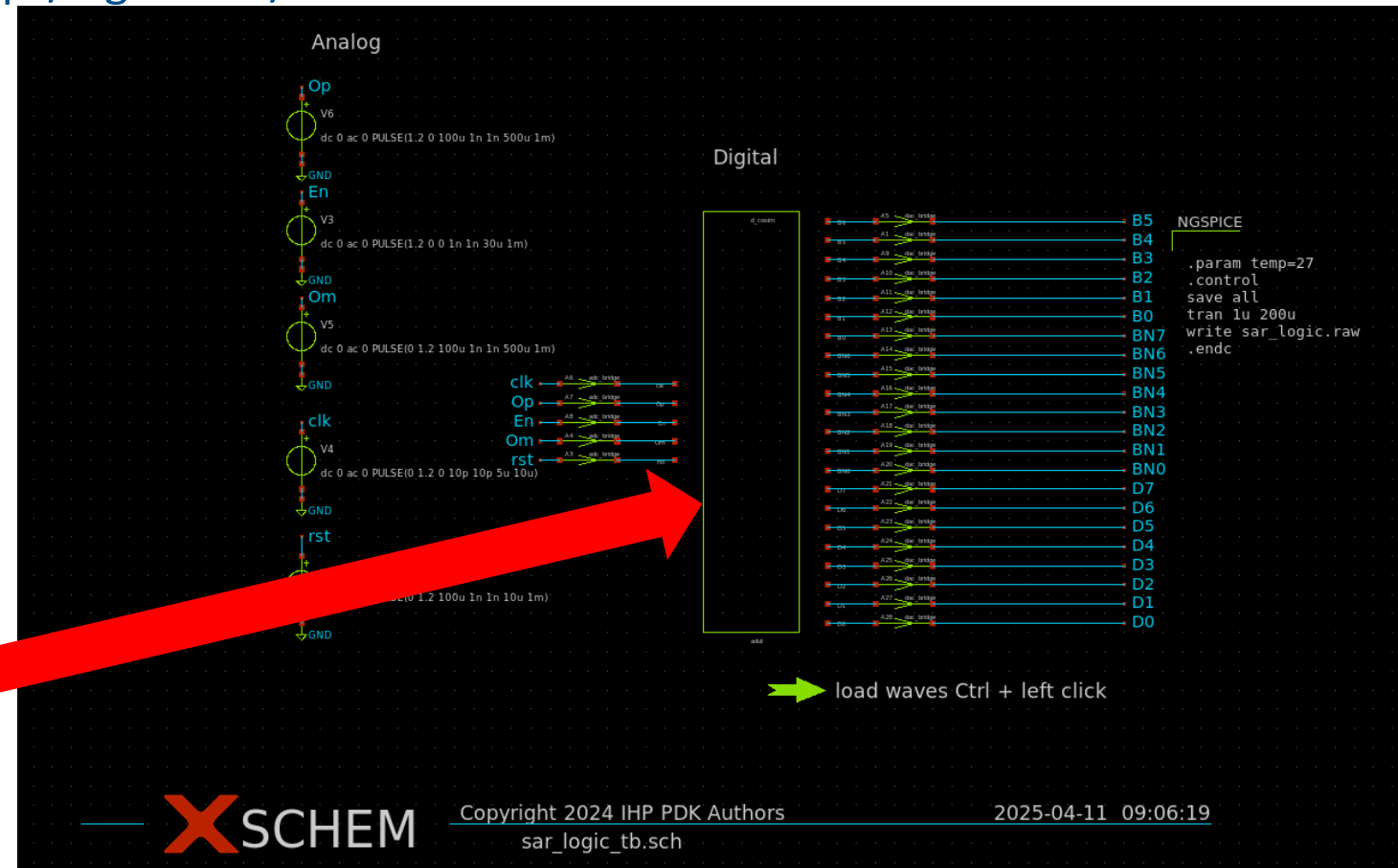
Navigate to the following path:

module_3_8_bit_SAR_ADC/part_2_digital_comps/algorithm/xschem

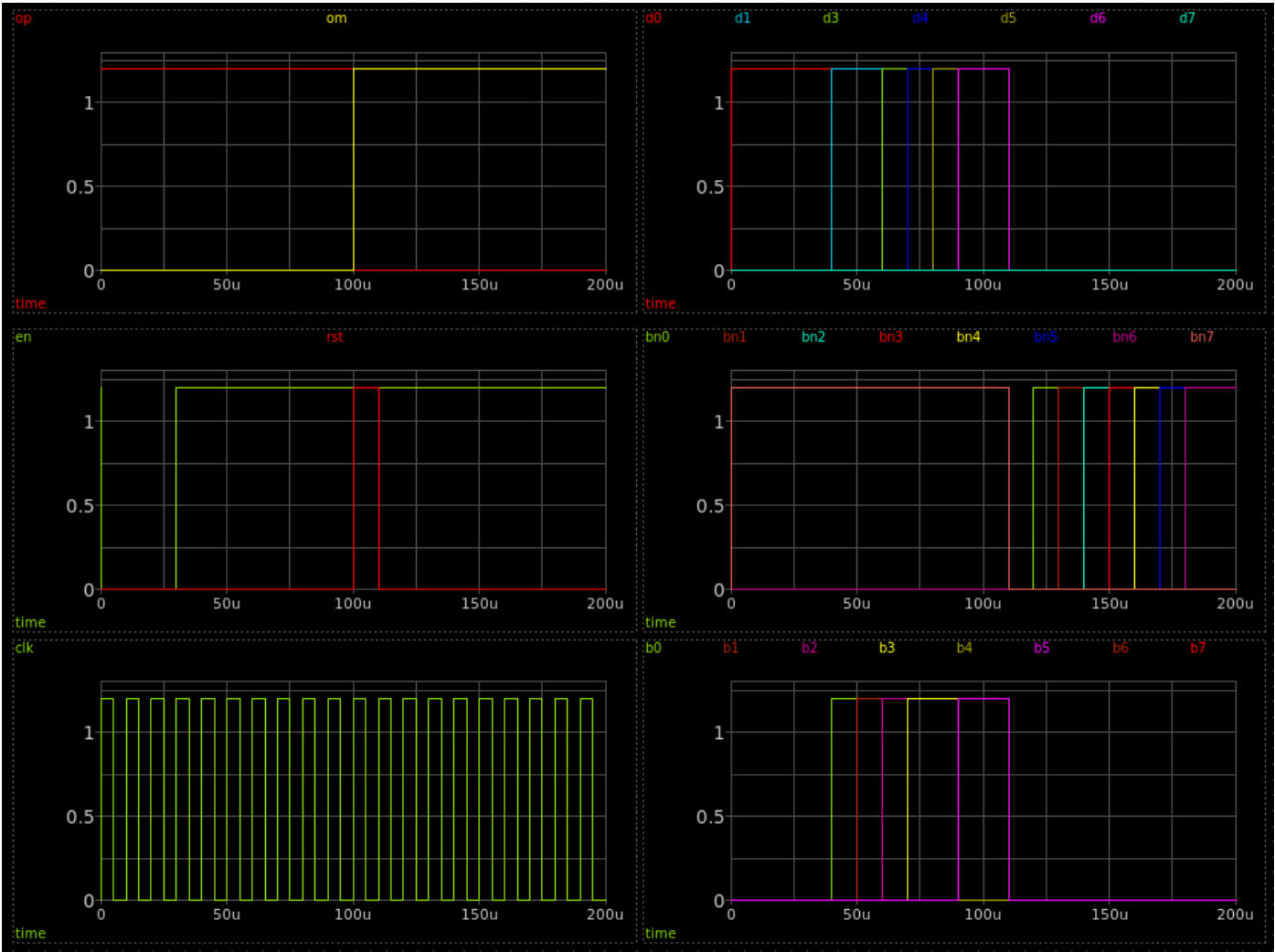
→ xschem git:(main) X xschem sar_logic_tb.sch

→ Simulate and confirm the operation of the digital block

```
name=adut
dut=dut
d_cosim_model= d_cosim
model=./sar_logic.so
```



Digital Block In Xschem





Leibniz Institute
for high
performance
microelectronics

Catching Up / Lunch Time !

A decorative graphic on the left side of the slide consisting of various colored lines (red, blue, grey) and small circles, resembling a circuit board or a network diagram.

Next session:
SAR Algorithm

Remaining Components



To realize the final schematic we have to create the following components:

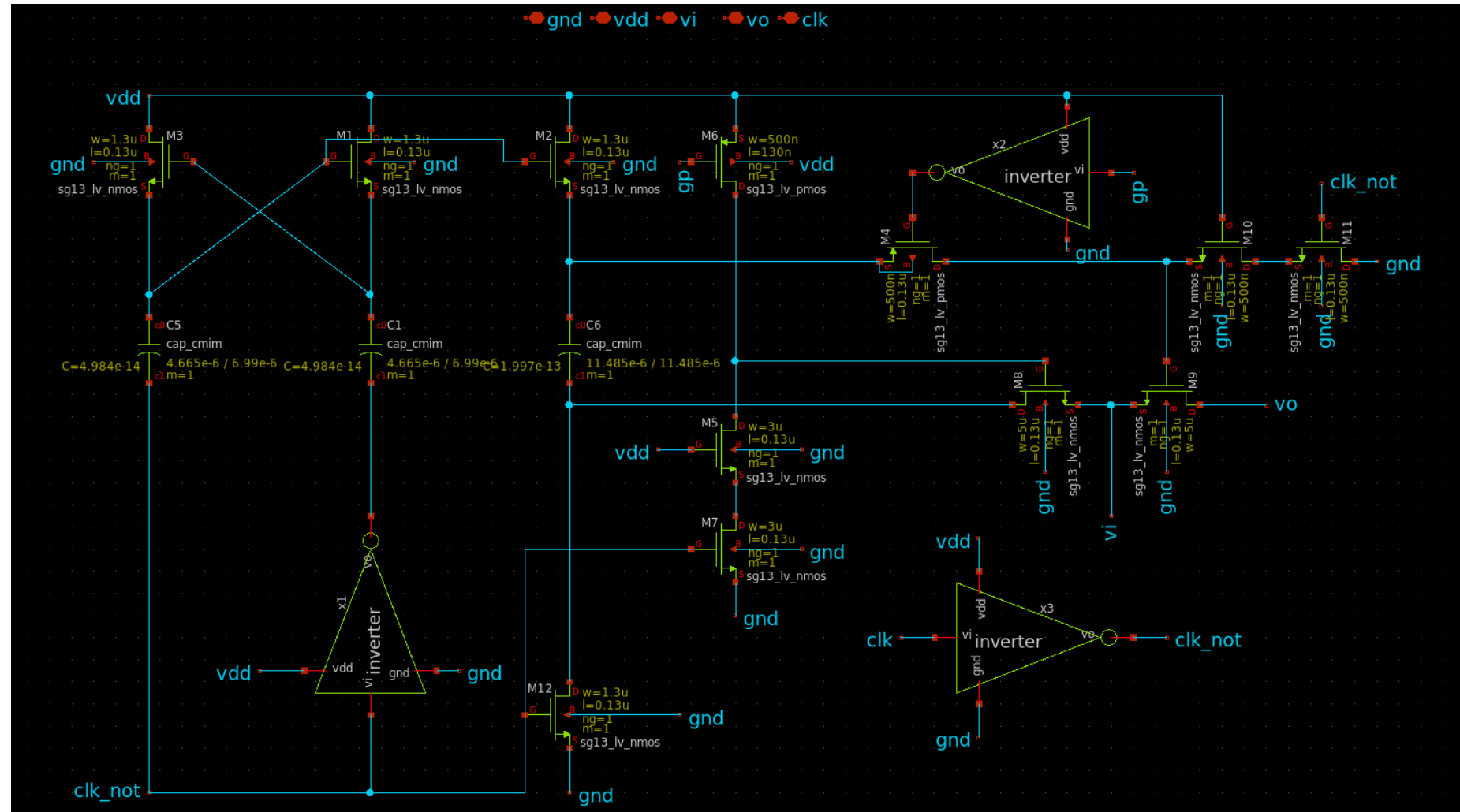
- 0 Bootstrap
- 0 Nand gate
- 0 Transmission gate

On the next couple of slides the circuits can be seen, its up to you to test them and confirm operation.

Bootstrap Switch



- The following schematic is quite large
- you may copy the schematic from the repository
- Good practice to create the symbol yourself, along with the testbench!
- For the testbench explanation refer to the markdown file under this part



Bootstrap Switch



NMOS Transistors (sg13_lv_nmos):

- M1: $W = 1.3 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M2: $W = 1.3 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M3: $W = 1.3 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M5: $W = 3.0 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M7: $W = 3.0 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M8: $W = 5.0 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M9: $W = 5.0 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M10: $W = 0.5 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M11: $W = 0.5 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M12: $W = 1.3 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$

PMOS Transistors (sg13_lv_pmos):

- M4: $W = 0.5 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$
- M6: $W = 0.5 \mu\text{m}$, $L = 0.13 \mu\text{m}$, $ng = 1$, $m = 1$

Capacitors (cap_cmim):

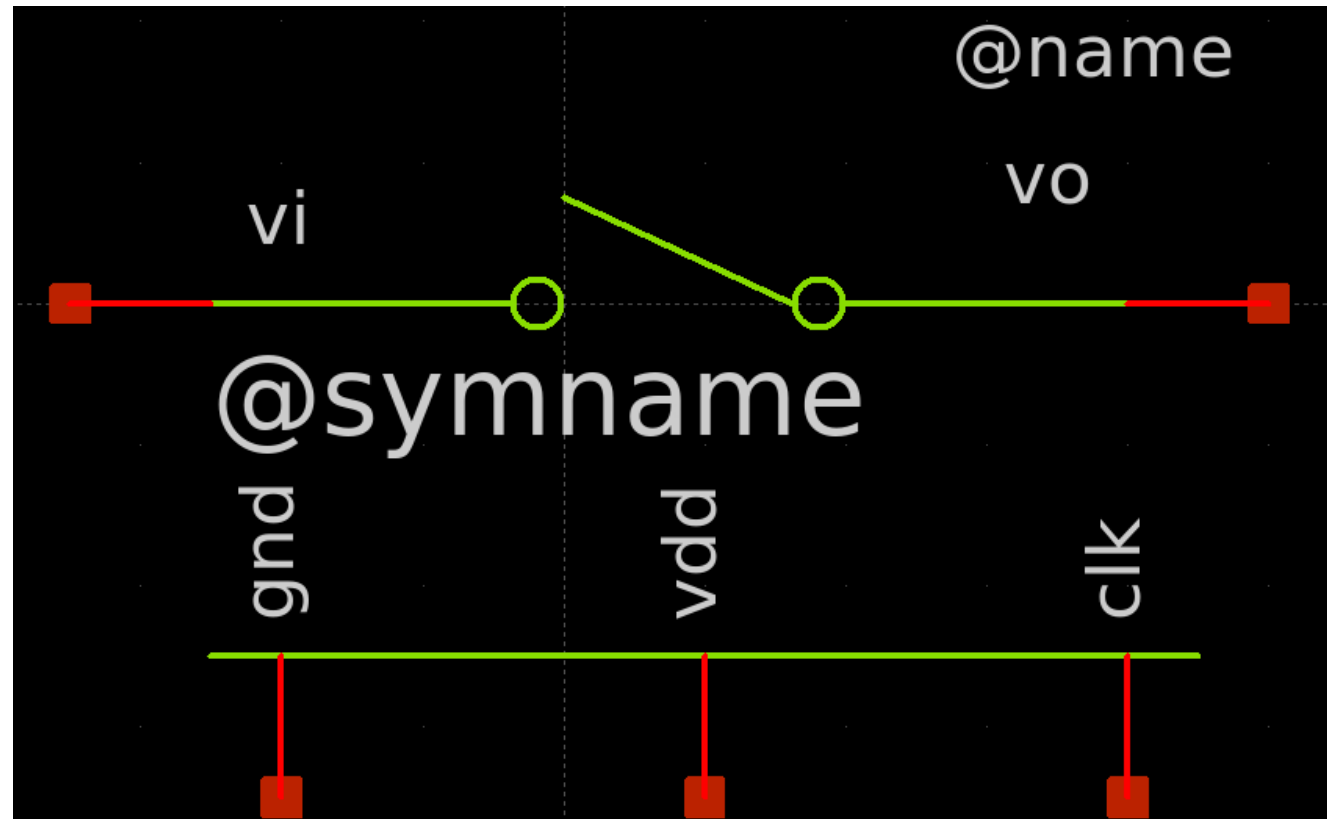
- C1: $W = 4.665 \mu\text{m}$ $L = 6.99 \mu\text{m}$ $m = 1$
- C5: $W = 4.665 \mu\text{m}$ $L = 6.99 \mu\text{m}$ $m = 1$
- C6: $W = 11.485 \mu\text{m}$ $L = 11.485 \mu\text{m}$ $m = 1$

For the inverter the reader is needed to design this use the following parameters **NMOS Transistors (sg13_lv_nmos):**

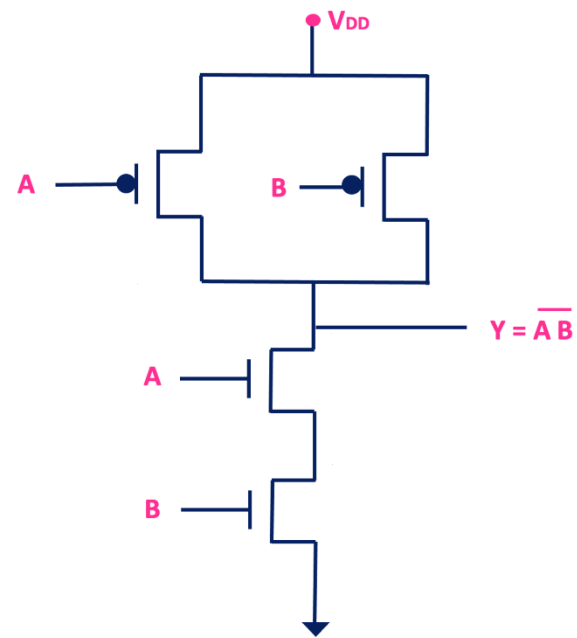
M1: $W = 1.3 \mu\text{m}$, $L = 0.4 \mu\text{m}$, $ng = 1$, $m = 1$

PMOS Transistors (sg13_lv_pmos):

- M2: $W = 1.3 \mu\text{m}$, $L = 0.8 \mu\text{m}$, $ng = 1$, $m = 1$

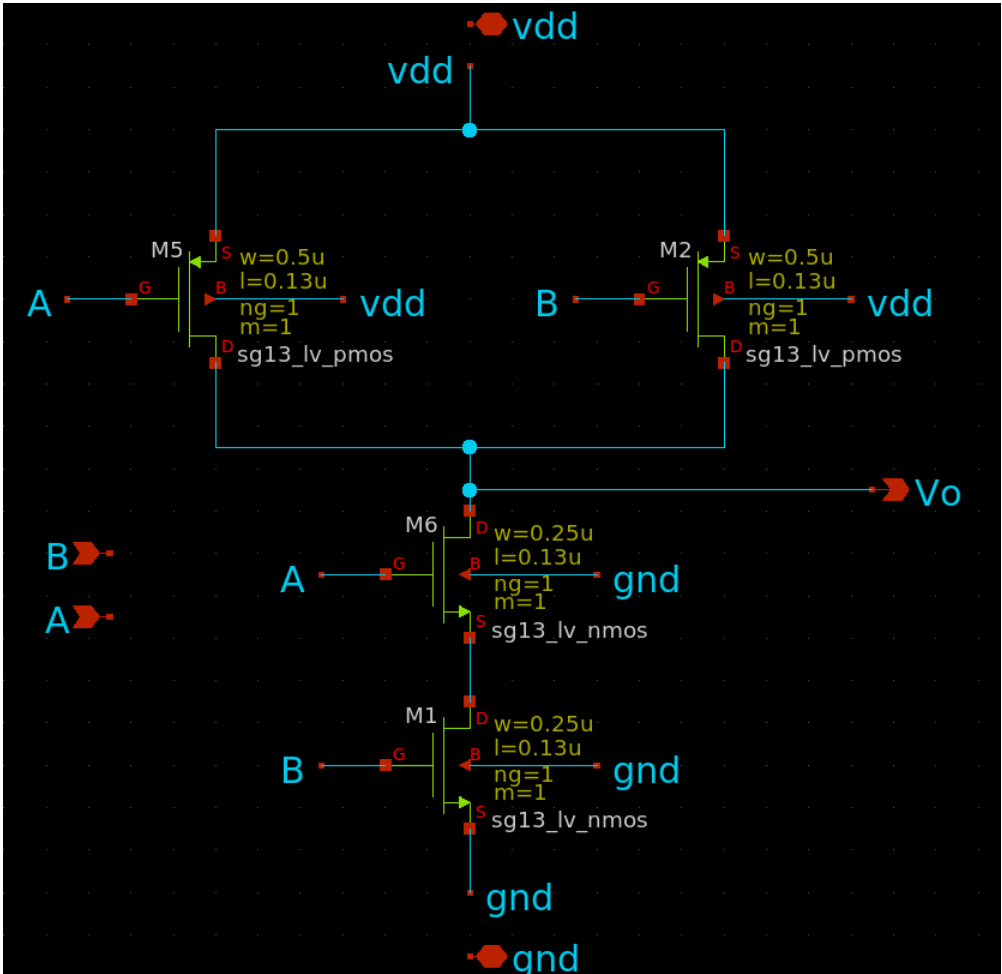
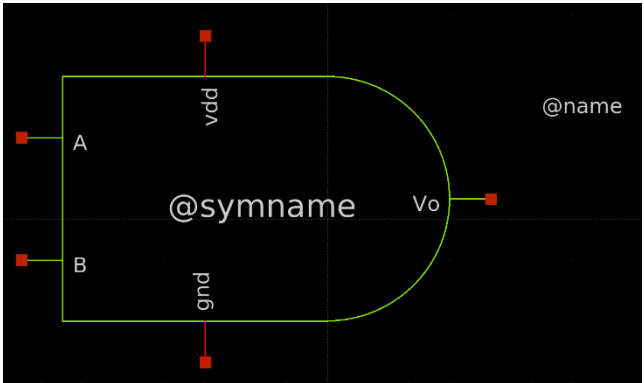


Nand Gate

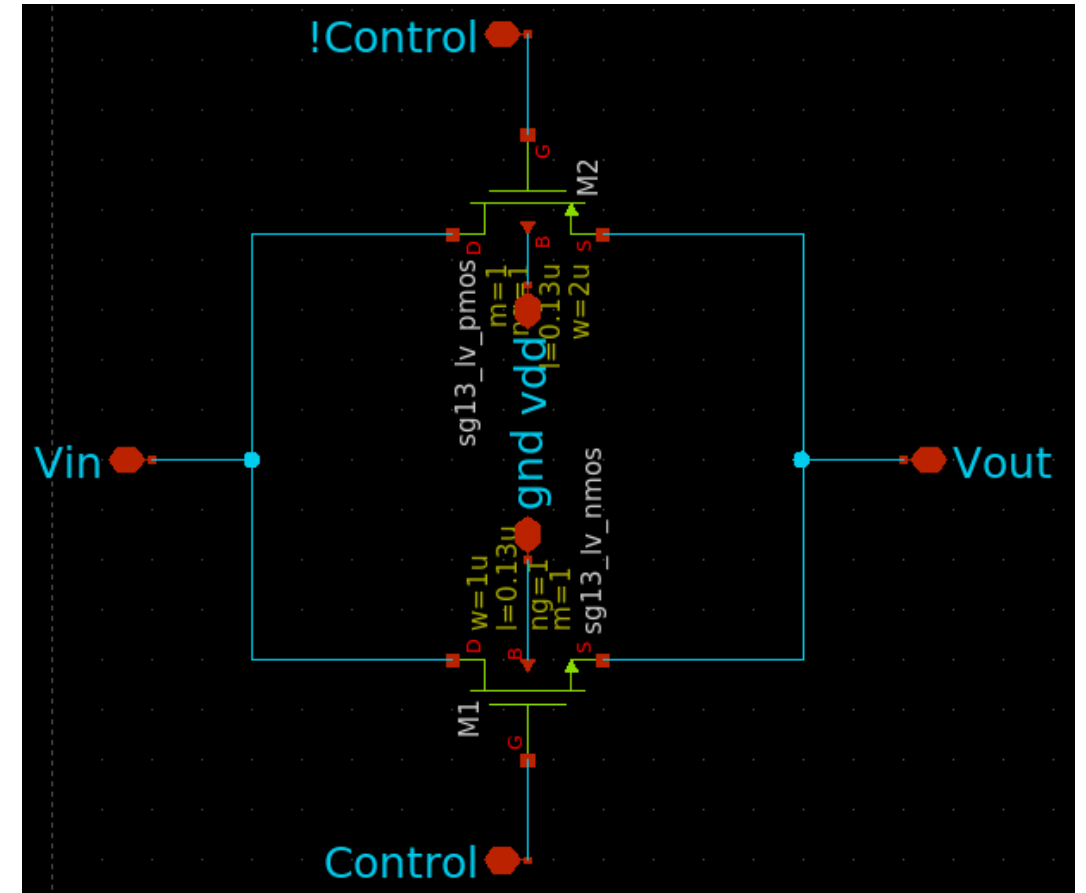
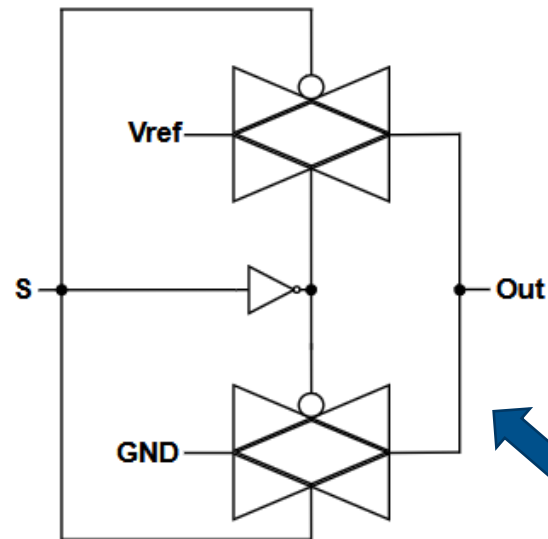
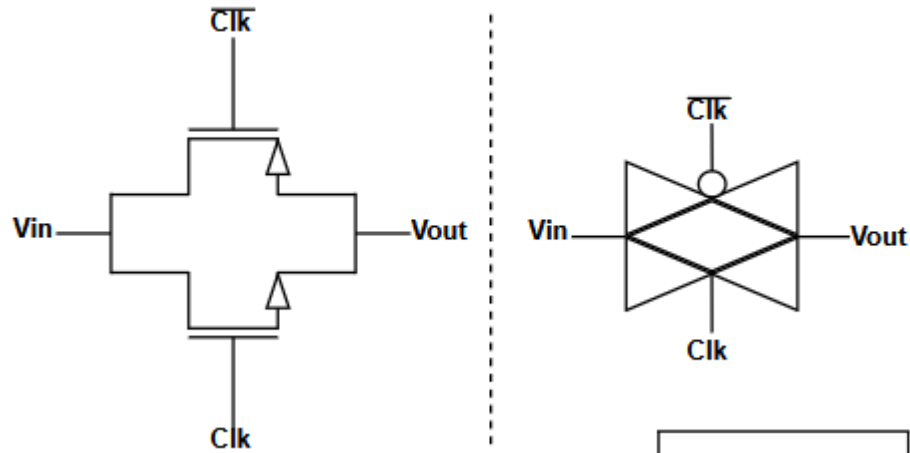


Truth Table

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



Transmission Gate



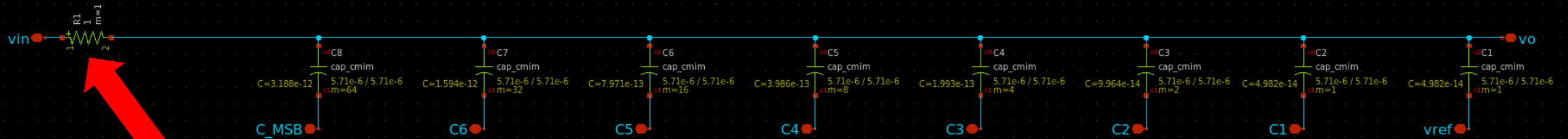
Part 3

A decorative circuit pattern is visible on the left side of the slide, featuring red and blue lines, circles, and a small square, resembling a printed circuit board layout.

Array Components

Reference: [modules/module_3_8_bit_SAR_ADC/part_3_array_components](#)

C-DAC (Capacitive DAC)



Why??

Capacitor Dimensions (cap_cmim):

C1 between vo and vref: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 1$

C2 between vo and C1: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 1$

C3 between vo and C2: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 2$

C4 between vo and C3: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 4$

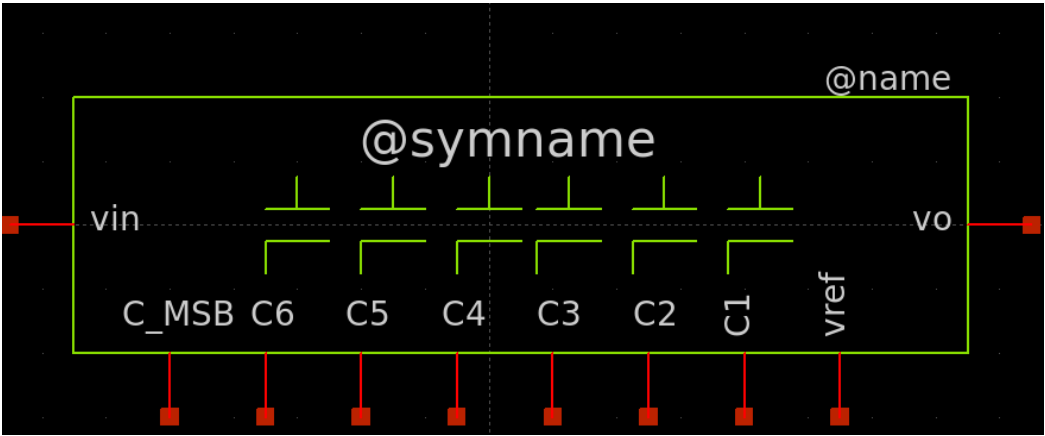
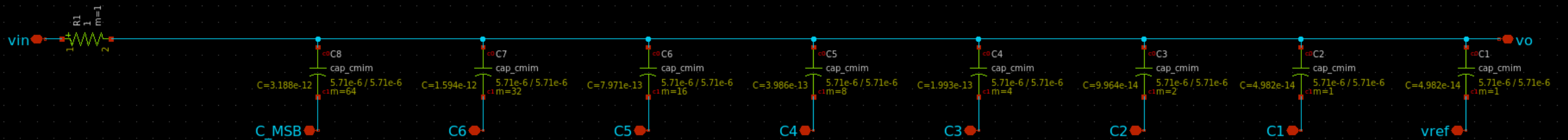
C5 between vo and C4: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 8$

C6 between vo and C5: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 16$

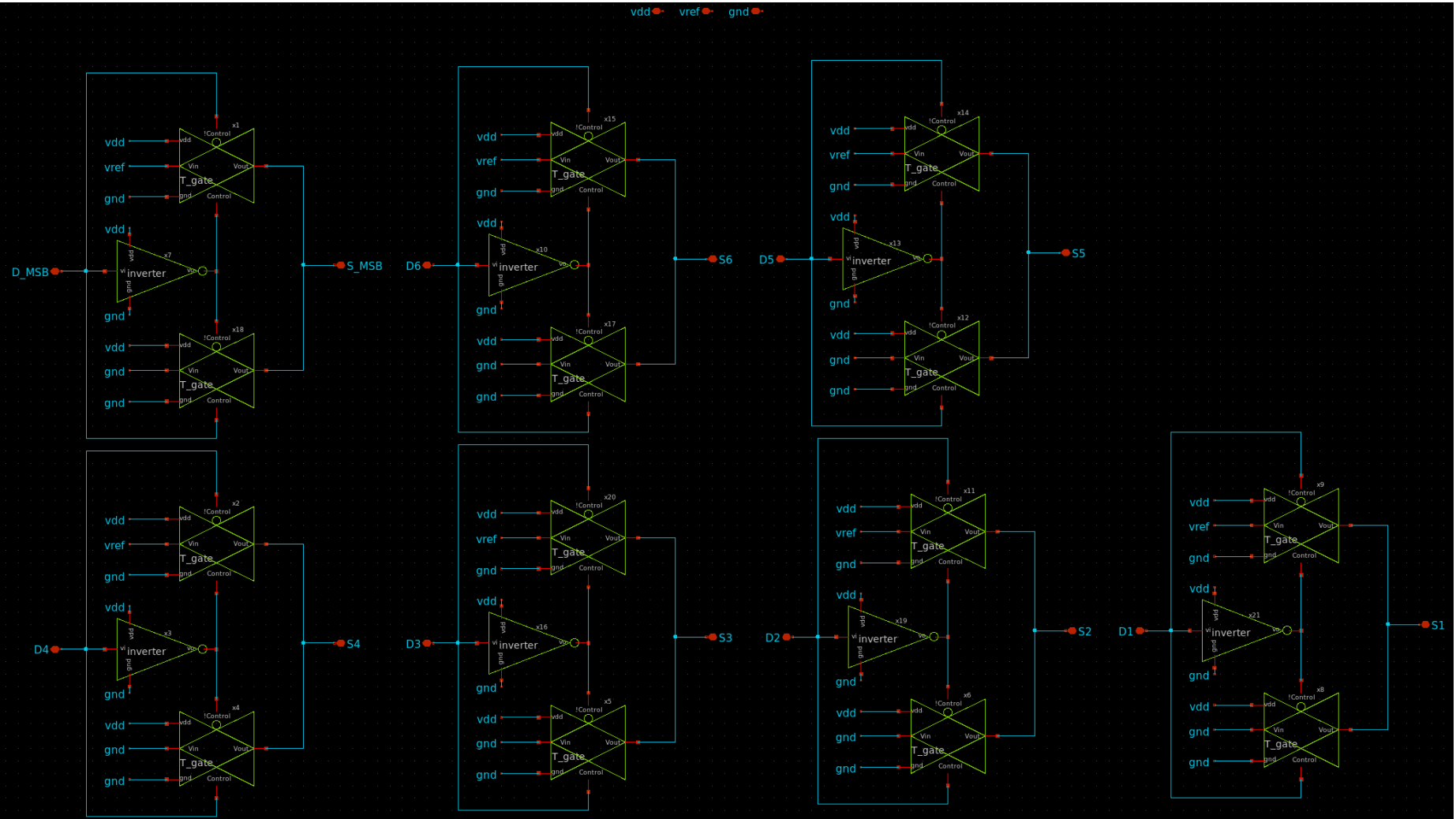
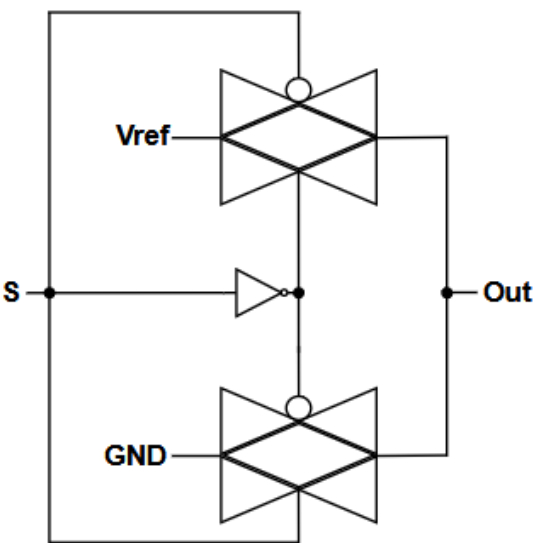
C7 between vo and C6: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 32$

C_MSB between vo and C_MSB: $W = 5.71 \mu\text{m}$, $L = 5.71 \mu\text{m}$, $m = 64$

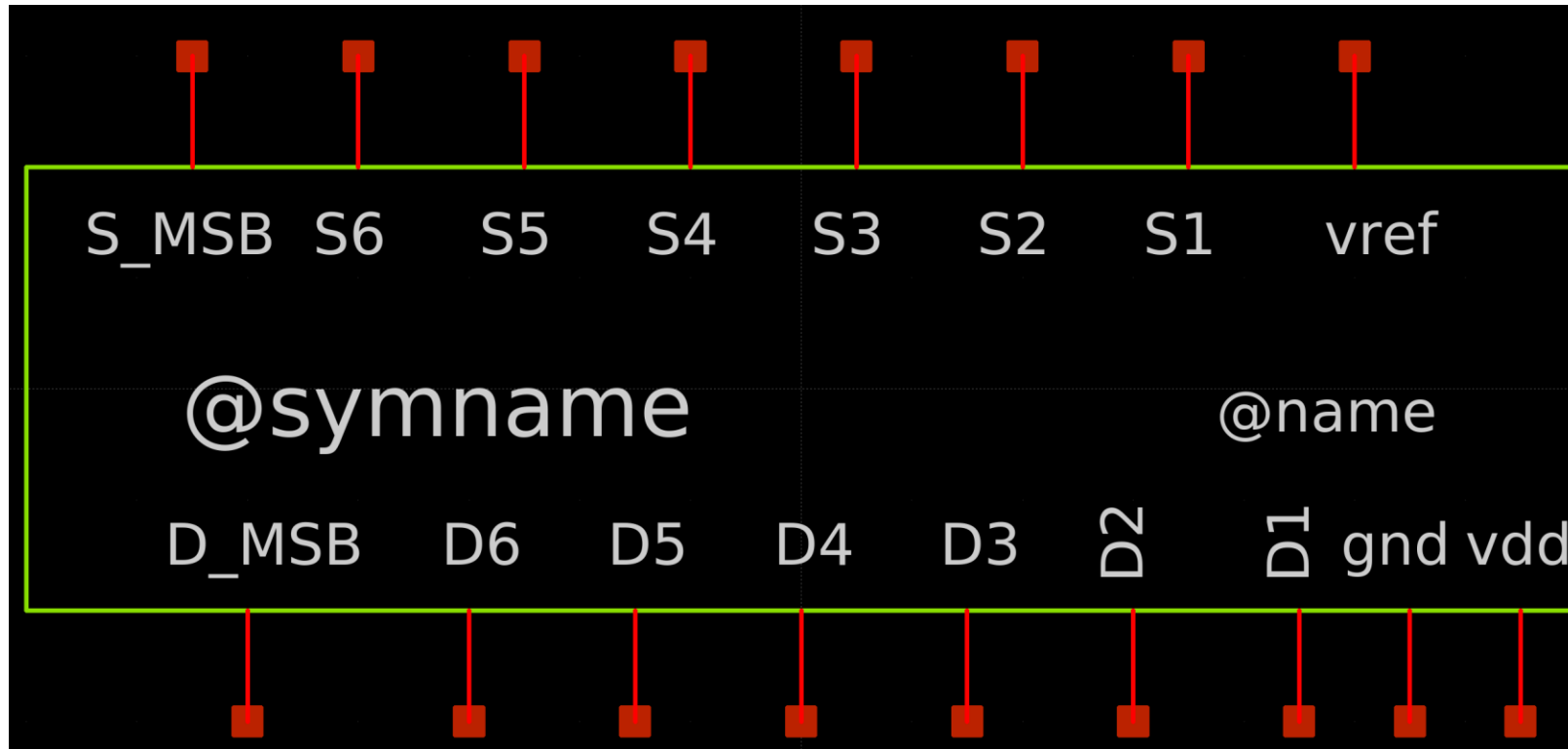
C-DAC (Capacitive DAC)



Switch Bank



Switch Bank



Part 4

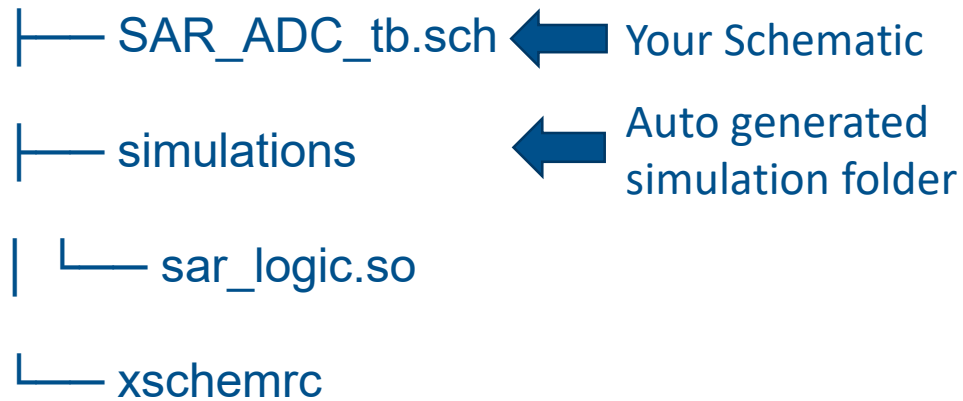
Complete SAR ADC

Reference: [modules/module_3_8_bit_SAR_ADC/part_4_SAR_ADC](#)

Importing SAR Algorithm

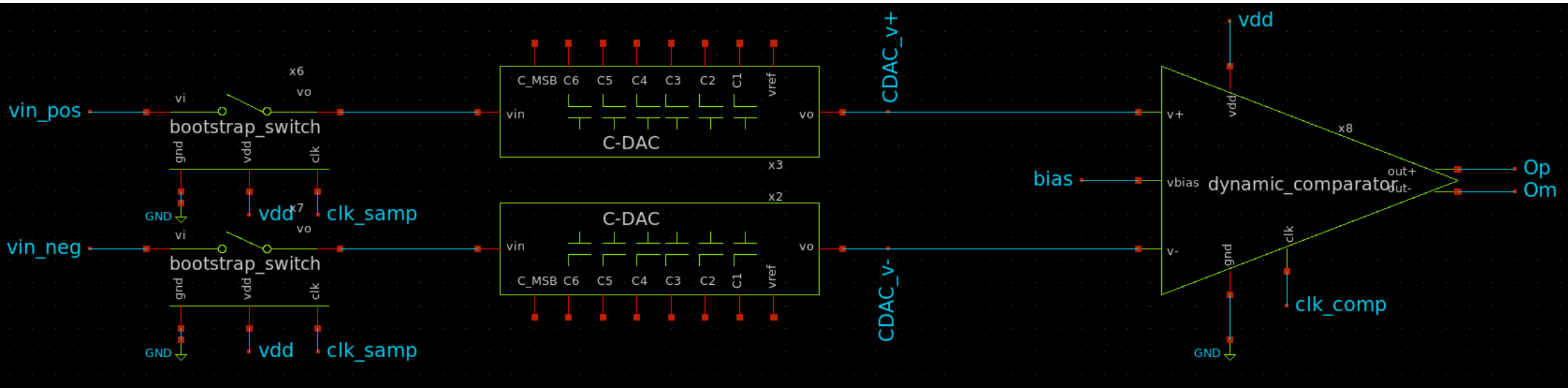


⚠ Note: Make sure to copy the shared object file (.so) and place it relative to the testbench, just like we did before. Create a simulation folder and include it there:

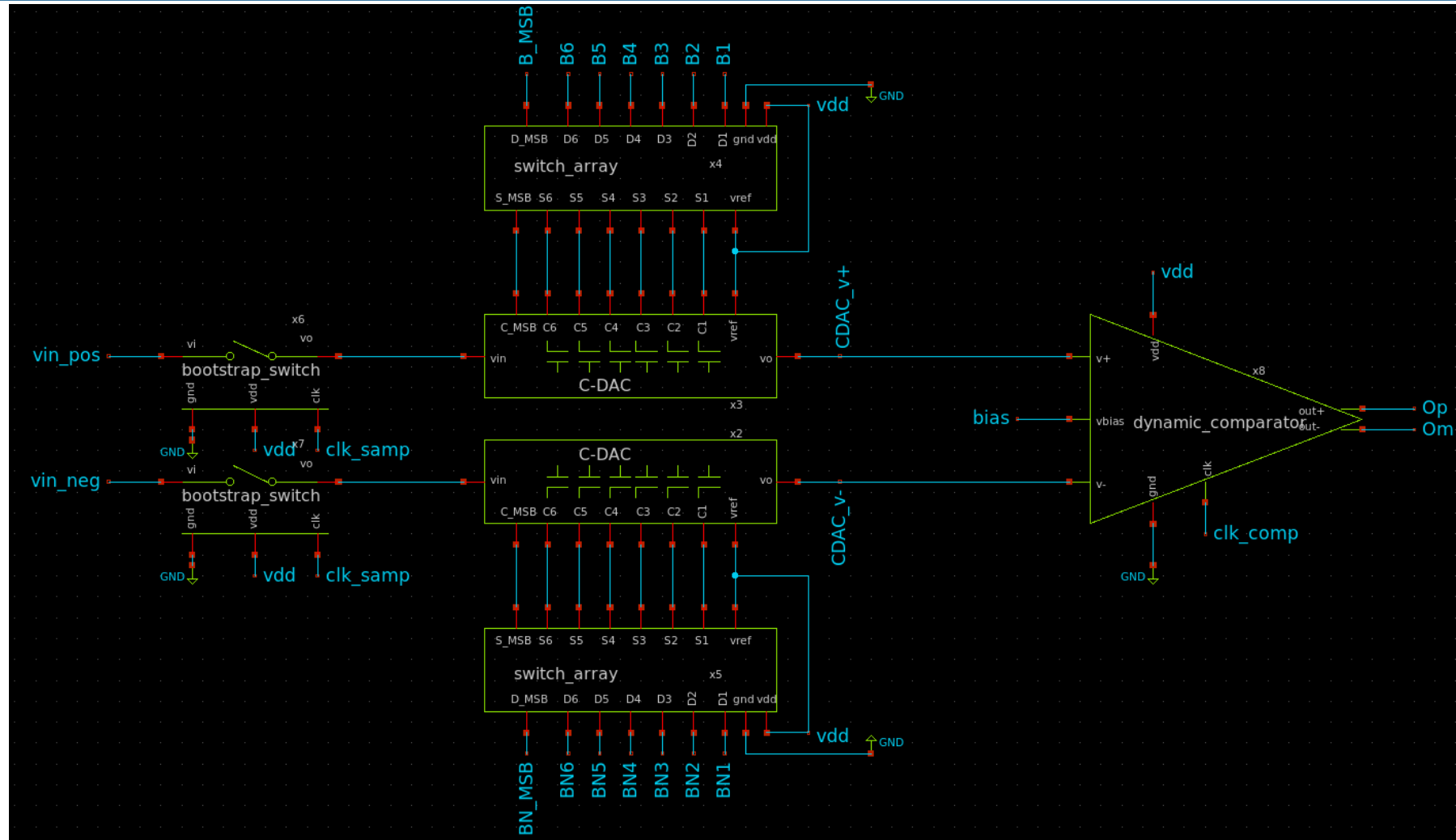


Make sure this works!

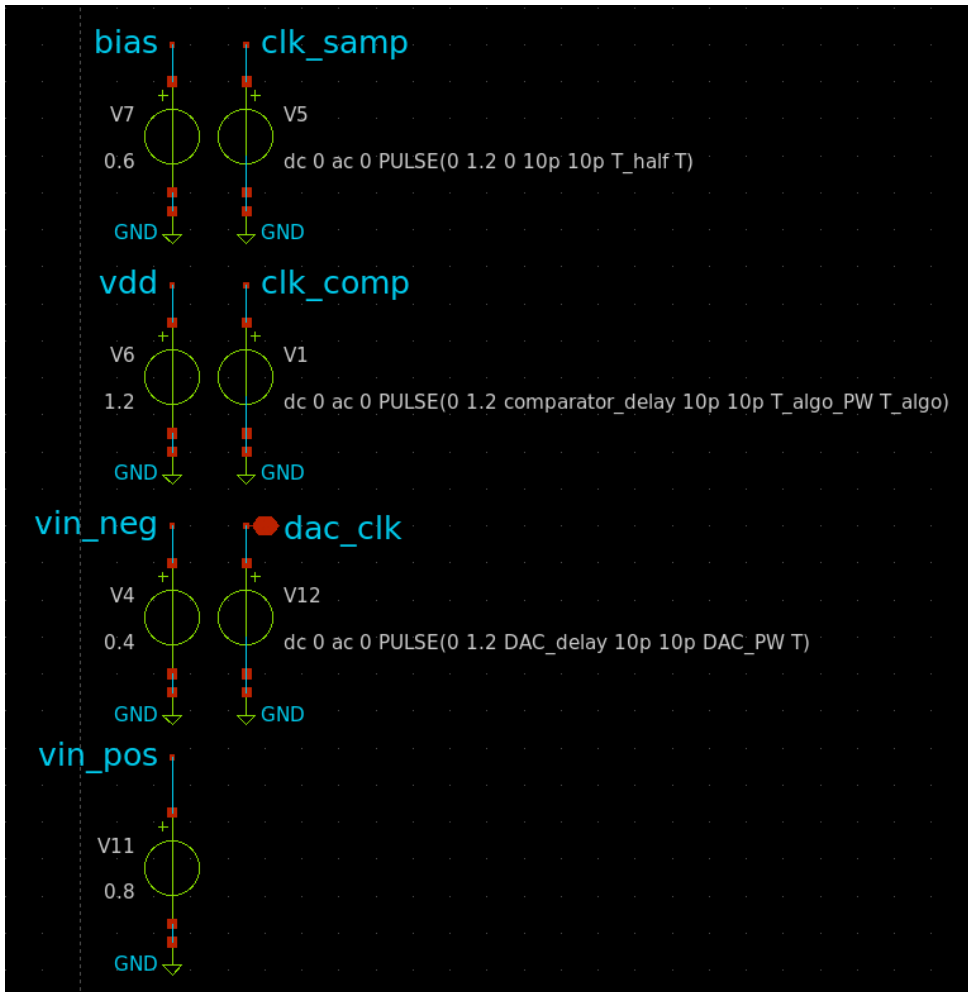
S/H, Comparator, and C-DAC



Switch Bank Incorporation



Testbench Setup



Signals:

bias → Simple Bias voltage

clk_samp → sampling clock for S/H and Reset

clk_comp → scaled clock signal for comparison cycles (covers all the bits within one sample window)

vdd → Power supply

vin_neg → negative input voltage

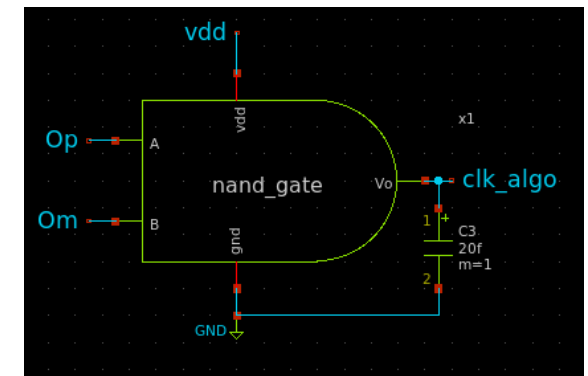
dac_clk → clock for external processing in python

vin_pos → positive input voltage

Op → positive output of comparator

Om → negative output of comparator

clk_algo → provides the “clock” for the SAR algorithm



Testbench Setup

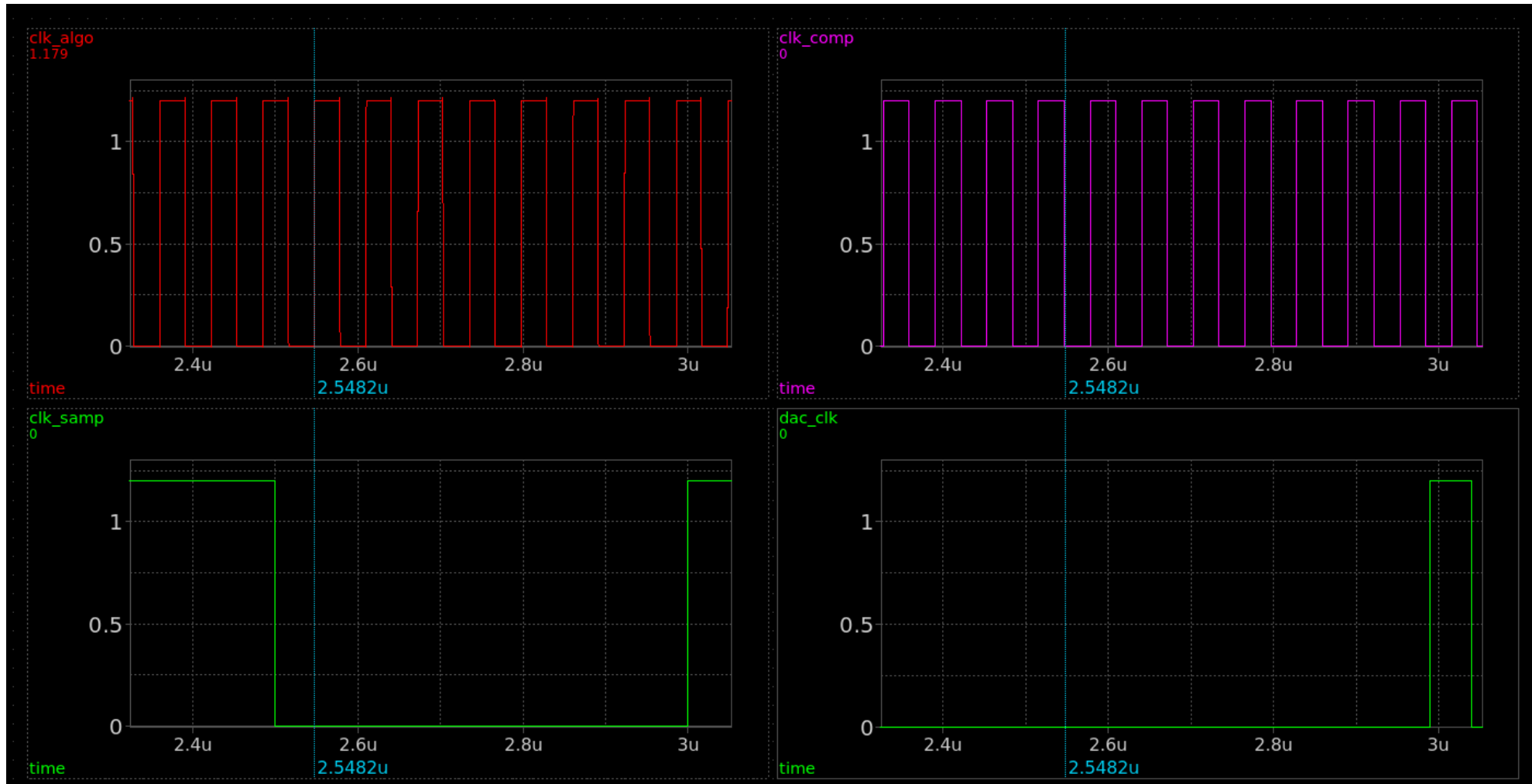


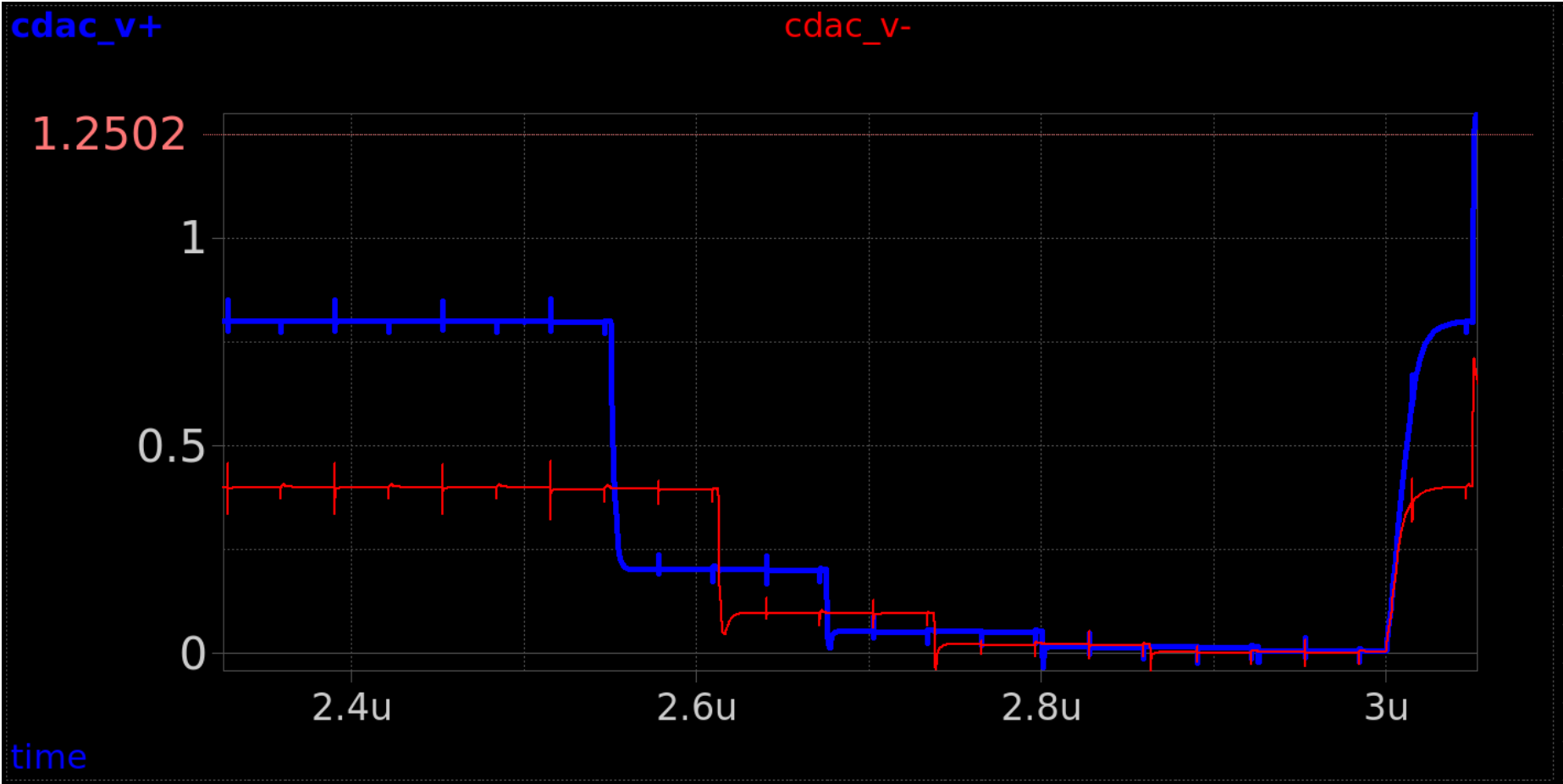
```
name=NGSPICE only_toplevel=true
value="
.param temp=27
.param T = 1u
.param T_half = T/2
.param T_algo = T/16
.param T_algo_delay = T/10
.param T_algo_PW = T/32
.param DAC_delay = 0.99*T
.param DAC_PW = T/20
.param comparator_delay = 0.328*T

.control
tran 1u 8u
let vin_diff = v(Vin_pos) - v(Vin_neg)
let comp_diff = v(op)- v(om)
write sar_adc_test.raw
.endc
"
```

```
name=MODEL only_toplevel=true
format="tclevel( @value )"
value=".lib cornerMOSlv.lib mos_tt
.lib $::SG13G2_MODELS/cornerCAP.lib cap_typ
"
```


Testbench Setup





Post-Processing Using Ideal DAC



```
"  
.param temp=27  
.param T = 1u  
.param T_half = T/2  
.param T_algo = T/16  
.param T_algo_delay = T/10  
.param T_algo_PW = T/32  
.param DAC_delay = 0.99*T  
.param DAC_PW = T/20  
.param comparator_delay = 0.328*T  
  
.control  
tran 1u 8u  
let vin_diff = v(Vin_pos) - v(Vin_neg)  
let comp_diff = v(op)- v(om)  
set wr_singlescale  
set wr_vecnames  
wrdata bit_data.txt D0 D1 D2 D3 D4 D5 D6 D7 vin_diff dac_clk  
.endc  
"
```

The extracted data comes directly from the SAR logic and includes:

The 8 output bits (D0–D7)

The differential input voltage (vin_diff)

The DAC clock signal (dac_clk), which serves as a virtual clock for reconstructing the analog output using an ideal DAC in Python.

Adjustment to save the data!

Post-Processing Script Path: modules\module_3_8_bit_SAR_ADC\part_4_SAR_ADC\scripting

Post-Processing Using Ideal DAC



```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Read the whitespace-separated data
df = pd.read_csv('../simulations/bit_data.txt', sep=r'\s+')

# Display the first few rows
df
```

```
[1]:
```

	time	D0	D1	D2	D3	D4	D5	D6	D7	vin_diff	dac_clk
0	0.000000e+00	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.000000e+00	0.0
1	1.000000e-13	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	4.787787e-09	0.0
2	2.000000e-13	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	9.575574e-09	0.0
3	4.000000e-13	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.915115e-08	0.0
4	8.000000e-13	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	3.830230e-08	0.0
...	...	...	...	...	...	...	...	...	...	...	...
601056	3.199908e-04	1.2	0.0	0.0	1.2	0.0	1.2	1.2	0.0	2.344200e-01	1.2
601057	3.199916e-04	1.2	0.0	0.0	1.2	0.0	1.2	1.2	0.0	2.344562e-01	1.2
601058	3.199933e-04	1.2	0.0	0.0	1.2	0.0	1.2	1.2	0.0	2.345284e-01	1.2
601059	3.199966e-04	1.2	0.0	0.0	1.2	0.0	1.2	1.2	0.0	2.346728e-01	1.2
601060	3.200000e-04	1.2	0.0	0.0	1.2	0.0	1.2	1.2	0.0	2.348242e-01	1.2

601061 rows x 11 columns

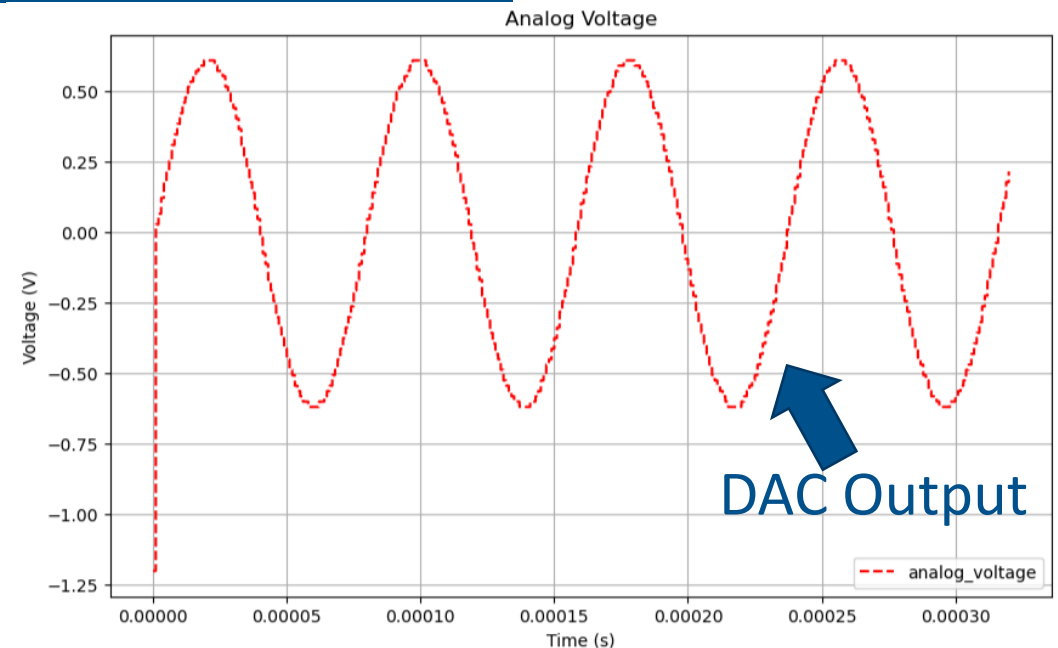
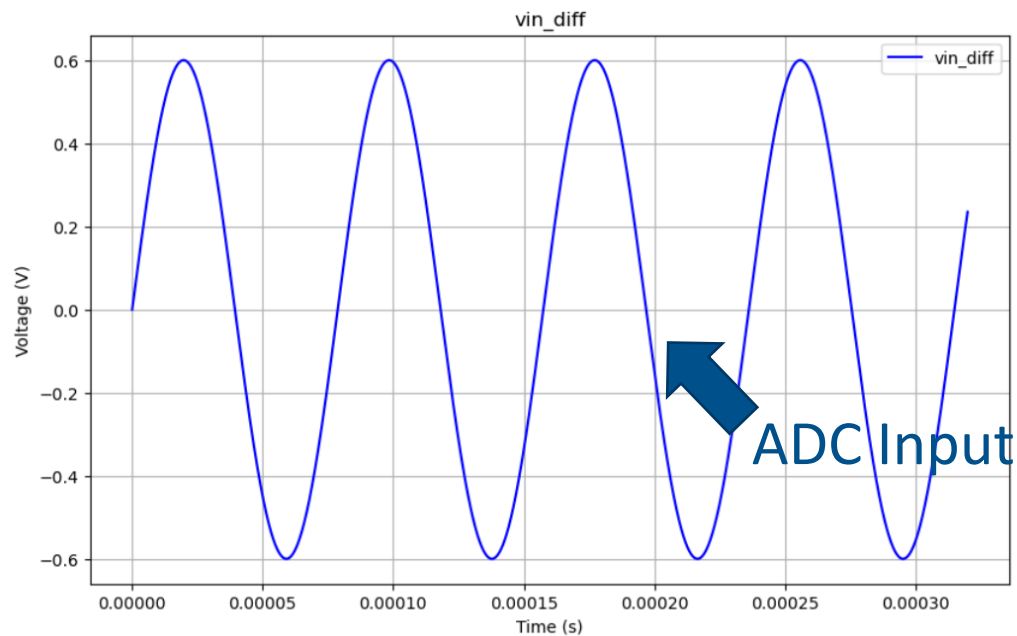
Use Your Virtual Env from the other tutorials ☺

Remaining Task



- 0 Apply a differential sinusoidal signal and recover it using the ideal DAC in the python script.
- 0 The plot below can be used as inspiration on what to expect and the guide on how to do this is available in the markdown file if you get stuck:

`module_3_8_bit_SAR_ADC/part_4_SAR_ADC/sar_adc.md`



Alternatively



- 0 **Analyze Performance Metrics:** Use the python script to assess some of the performance metrics of the ADC (ENOB, SNR, DNL etc...)
- 0 **Catch-Up:** Work on material from today or the other days to catch up or clarify points that wasn't clear
- 0 **Relax** 😊